BoRLang
& BoRFlow

The Complete Manual

A hands-on guide to the scripting language and agentic workflow engine for machine learning, vector search, and block-based automation.

L.L.B && D.B

version 3.1

Preface

This book is the official and complete manual for BoRLang and its workflow engine BoRFlow, together released as version 3.1. It is intended for programmers who want to use the tool, not for those who want to understand its internals — for the latter, the source code itself is small enough to read directly.

The manual is organized into twelve chapters. Chapters 1 through 5 cover the language: installation, primitive types, control flow, functions, and collections. Chapter 6 is a reference dump of the complete standard library. Chapters 7 through 10 walk through the specialized modules — machine learning, vector search, persistent memory, and quantum-inspired transforms. Chapter 11 introduces BoRFlow, the agentic workflow engine that is the defining feature of version 3. Chapter 12 is a cookbook of ready-to-run recipes.

Every code sample in the book has been tested against the official interpreter. If you encounter one that does not behave as described, it is a bug in the documentation (or the language) and you should file an issue.

The book is meant to be read in order the first time, then used as a reference afterward. Chapter 6 (the reference dump) and chapter 12 (the cookbook) are the pages you are most likely to come back to.

Milan, 2026

Contents

Preface 2

Contents 3

1.1 Design philosophy . . . 8
1.2 Comparison with other tools . . . 8
1.3 Installing BoRLang . . . 8
1.4 The project layout . . . 9
1.5 Your first BoRLang program . . . 9
1.6 The REPL . . . 9
1.7 What you will learn . . . 10
2.1 Comments . . . 11
2.2 Primitive types . . . 11
2.3 Variables and assignment . . . 12
2.4 Arithmetic operators . . . 12
2.5 Comparison operators . . . 13
2.6 Logical operators . . . 13
2.7 String concatenation . . . 13
2.8 Reserved keywords . . . 13
2.9 Semicolons and newlines . . . 14
3.1 The if statement . . . 15
3.2 Truthiness . . . 16
3.3 The while loop . . . 16
3.4 The for-in loop . . . 16
3.5 Nesting loops and conditionals . . . 17
3.6 Missing features . . . 17
4.1 Defining functions . . . 18
4.2 Multiple return paths . . . 18
4.3 Scoping rules . . . 19
4.4 Closures . . . 19

4.5 Stateful closures . . . 19
4.6 Higher-order functions . . . 20
4.7 Recursion . . . 20
4.8 Anonymous calls and pipelines . . . 20
5.1 Lists . . . 21
5.2 Dictionaries . . . 21
5.3 Strings in depth . . . 22
5.4 Regular expressions . . . 22
5.5 The len function . . . 23
6.1 I/O and conversion . . . 24
6.2 Strings . . . 24
6.3 Math . . . 25
6.4 Collections . . . 25
6.5 JSON . . . 25
6.6 File I/O . . . 25
6.7 System and time . . . 25
6.8 Crypto and encoding . . . 26
6.9 Machine learning (chapter 7) . . . 26
6.10 Vector store (chapter 8) . . . 26
6.11 Memory (chapter 9) . . . 26
6.12 Quantum-inspired (chapter 10) . . . 26
6.13 BoRFlow (chapter 11) . . . 27
7.1 Linear regression . . . 28
7.2 K-means clustering . . . 28
7.3 Principal component analysis . . . 29
7.4 Normalization . . . 29
7.5 Activation functions . . . 30
7.6 Statistical aggregates . . . 30
7.7 Cosine similarity . . . 30
7.8 Putting it together: a tiny classifier . . . 30
7.9 Worked example: customer segmentation . . . 31
7.10 Combining ML with flows . . . 32
8.1 The vector store abstraction . . . 33
8.2 Indexing documents . . . 33
8.3 Searching the store . . . 33

8.4 Building a minimal RAG pipeline . . . 34
8.5 FAISS backend . . . 34
8.6 Semantic embeddings . . . 35
9.1 Creating a memory store . . . 36
9.2 Saving and recalling . . . 36
9.3 Forgetting . . . 37
9.4 Introspection . . . 37
9.5 A conversational agent's memory . . . 37
9.6 When to use and not use memory . . . 37
10.1 Amplitude encoding . . . 38
10.2 Measurement . . . 38
10.3 Truncated SVD compression . . . 39
10.4 Dictionary coding (compress_zh) . . . 39
10.5 Real compression with zlib . . . 39
11.1 The mental model . . . 41
11.2 Building your first flow . . . 41
11.3 Block types . . . 41
11.4 Context and templating . . . 42
11.5 Branching with conditions . . . 42
11.6 Looping . . . 43
11.7 Calling LLMs . . . 43
11.8 HTTP requests . . . 43
11.9 Execution trace . . . 44
11.10 Serializing flows . . . 44
11.11 Custom action handlers . . . 44
12.1 Retrieval-augmented question answering . . . 45
12.2 Daily data pipeline . . . 45
12.3 Sentiment-gated support triage . . . 46
12.4 Conversational memory . . . 46
12.5 Simple linear-regression benchmark . . . 47
12.6 Document classifier . . . 47
12.7 Best practices . . . 48
12.8 Closing thoughts . . . 48
11A.1 Error handling and retries . . . 49
11A.2 Composing sub-flows . . . 50

11A.3 Streaming versus batch . . . 50
11A.4 Rate limiting . . . 51
11A.5 Persisting and replaying flows . . . 51
11A.6 Observability with trace analysis . . . 52
11A.7 Testing flows . . . 52
11A.8 Caching expensive blocks . . . 53
11A.9 Dynamic flow construction . . . 54
11A.10 Debugging flows . . . 54
11A.11 Measuring agent quality . . . 55
13.1 Project 1: Text analyzer . . . 57
13.2 Project 2: Personal notes agent . . . 61
13.3 Project 3: Folder semantic search . . . 64
13.4 Project 4: Automated research assistant . . . 66
13.5 Project 5: Multi-agent delegator . . . 69
13.6 What to build next . . . 71
14.1 Project 6: Log anomaly detector . . . 73
14.2 Project 7: Chatbot with long-term memory . . . 76
A.1 Literal forms . . . 79
A.2 Statements . . . 79
A.3 Operators . . . 79
A.4 Running programs . . . 79
D.1 Running on a server . . . 85
D.2 Secrets and credentials . . . 85
D.3 Monitoring agent health . . . 86
D.4 Common errors and fixes . . . 86
- ModuleNotFoundError: No module named numpy . . . 86
- llm node 'x' failed: could not reach Ollama at 86
- SyntaxError near 'end' . . . 86
- RuntimeError: maximum recursion depth . . . 86
- KeyError: 'X' . . . 87
- Flow seems to hang forever . . . 87

D.5 Frequently asked questions . . . 87
- Is BoRLang production-ready? . . . 87
- Can I embed BoRLang in another Python app? . . . 87
- Why not use Python directly? . . . 87
- Why not use LangChain? . . . 87

Does BoRLang support async? . . . 88
How do I share a flow with a collaborator? . . . 88
Where is the source code? . . . 88
D.6 Where to go from here . . . 88
E.1 Why 'end' instead of indentation . . . 89
E.2 Why dynamic typing . . . 89
E.3 Why no classes . . . 89
E.4 Why flow-based agents . . . 90
E.5 Why not compile to Python . . . 90
E.6 Why Python as the host language . . . 90
E.7 What is deliberately missing . . . 91
E.8 What the future looks like . . . 91
F.1 Data manipulation patterns . . . 92
F.2 Flow construction patterns . . . 94
F.3 Error handling patterns . . . 96
F.4 I/O patterns . . . 97
F.5 Agent patterns . . . 98
F.6 Testing patterns . . . 99

1 Introduction & Installation

BoRLang is a small, practical scripting language designed for machine learning experimentation, vector search, and building agentic workflows. It looks like Python, scopes like Lua, and ships with a standard library of over 100 real implementations backed by NumPy, scikit-learn, and SQLite. BoRFlow, bundled with the same package, extends BoRLang with an n8n-style workflow engine: define blocks, connect them into a DAG, execute.

This manual is written for three audiences: the ML engineer who wants a lightweight DSL to prototype pipelines, the web developer curious about agentic patterns, and the student learning how a real interpreter works from the inside. Every code sample in this book runs on the official BoRLang 3.1 interpreter with no additional dependencies beyond NumPy and scikit-learn.

1.1 Design philosophy

BoRLang was designed around three principles. First, **honesty**: every function in the standard library does exactly what its name suggests, with no stubs or simulated outputs. If a function cannot perform its task (for example, an LLM call when Ollama is not running), it raises a clear error rather than returning fake data. Second, **minimalism**: the core interpreter has zero runtime dependencies outside the Python standard library; the ML-oriented standard library adds NumPy and scikit-learn, nothing else. Third, **programmable agents**: rather than providing a visual-only workflow tool, BoRFlow exposes every flow primitive as a regular BoRLang function so that agents can be defined, versioned, and modified as source code.

1.2 Comparison with other tools

BoRLang occupies a space that is currently served by disparate tools. If you want quick ML scripting, you reach for Python with NumPy and scikit-learn; if you want agentic pipelines, you reach for n8n or RAGFlow; if you want a small embedded scripting language, you reach for Lua or Tcl. BoRLang combines all three in a single package weighing less than 50 KB. It is not intended to replace any of them for large-scale production use. Instead, it shines in prototypes, educational contexts, and small projects where installing a heavy toolchain would be overkill.

1.3 Installing BoRLang

BoRLang is distributed as a single ZIP archive or a GitHub repository. The core interpreter needs nothing beyond Python 3.10+. The standard library requires two optional packages:

```
# Minimum install (everything works)
pip install numpy scikit-learn

# Optional: fast vector index
pip install faiss-cpu

# Optional: semantic embeddings
pip install sentence-transformers
```

Once the ZIP is unpacked, verify the installation:

```
cd borlang_v3_1/
python borlang.py                    # starts the REPL
python test_borlang.py               # should print 38/38 passed
python test_flow.py                  # should print 15/15 passed
```

> **NOTE.** If your system ships with Python 3.9 or older, BoRLang will refuse to start because it relies on union-type syntax introduced in 3.10. Install a newer interpreter via *pyenv*, *conda*, or your OS package manager.

1.4 The project layout

After extracting the ZIP, you will see the following files and directories:

```
borlang_v3_1/
■■■ borlang.py              # Interpreter: lexer + parser + evaluator
■■■ borlang_stdlib.py       # Standard library (100+ functions)
■■■ borlang_flow.py         # BoRFlow workflow engine
■■■ test_borlang.py         # Core test suite (38 tests)
■■■ test_flow.py            # Flow engine tests (15 tests)
■■■ playground.html         # Browser code editor
■■■ flow_designer.html      # Visual flow designer (n8n-style)
■■■ README.md               # Quick reference
■■■ LICENSE                 # MIT license
■■■ requirements.txt        # Python dependencies
■■■ examples/
    ■■■ demo_v3.bor         # Sixteen-module walkthrough
    ■■■ agent_rag.bor       # Agentic RAG pipeline
```

1.5 Your first BoRLang program

Open a file called **hello.bor** in any text editor and type:

```
# My first BoRLang program
name = "world"
print("Hello, " + name + "!")
```

Save the file and run it from the terminal:

```
python borlang.py hello.bor
# Hello, world!
```

Congratulations. You have just used all three pillars of the language: a comment (lines beginning with #), a variable assignment, and a built-in function call. The rest of this manual expands on each of these ideas in detail.

1.6 The REPL

Running **python borlang.py** without any file arguments drops you into an interactive loop. Type expressions and see their results immediately:

```
$ python borlang.py
BoRLang v3.1 — type 'exit' to quit
bor> 2 + 3 * 4
14
bor> name = "Borty"
bor> print("hi " + name)
```

```
hi Borty
bor> exit
```

The REPL supports multi-line input: if you type **def greet(name)** and press enter, the prompt changes to an ellipsis until you close the block with **end**. This makes it convenient to sketch short functions before committing them to a file.

1.7 What you will learn

Chapters 2 through 5 teach the language itself: data types, variables, expressions, control flow, functions, closures, and the three core collection types (lists, dictionaries, strings). Chapter 6 reference-dumps the entire standard library so you always know what is available without reading the source code.

Chapters 7 through 10 cover the non-core modules that make BoRLang useful for real work: machine learning, vector search, persistent memory, and the quantum-inspired compression primitives that originally motivated the language. Chapter 11 is the longest in the book and introduces BoRFlow — the agentic workflow engine that turns BoRLang into a platform for building small language model agents. Chapter 12 is a cookbook of real, working recipes that combine everything.

2 Language Basics

This chapter walks through the atomic elements of BoRLang: the types of values you can manipulate, how to name them, how to combine them with operators, and how the parser treats whitespace and comments. If you know Python, you will find most of this familiar; the differences are noted in context.

2.1 Comments

BoRLang supports two comment syntaxes. A **hash mark (#)** begins a single-line comment in the Python tradition; a pair of **forward slashes (//)** does the same in the C tradition. Both run to the end of the line and are completely ignored by the parser. There is currently no multi-line or block-comment syntax — if you want to comment out a paragraph, prefix every line with a hash.

```
# This is a comment.
// This is also a comment.
x = 42           # trailing comments work too
y = 17           // and with the other style
```

2.2 Primitive types

BoRLang has five primitive value types. Each literal in the source code produces a value of one of these types, which the interpreter stores and manipulates at runtime.

Integers. Whole numbers, written in decimal notation. Internally represented using Python's arbitrary-precision int type, so overflow is never a concern.

```
age = 30
large = 999999999999999999
negative = -42
```

Floats. Numbers containing a decimal point. Backed by IEEE 754 double-precision floats, so the usual rounding caveats apply: 0.1 + 0.2 will not equal 0.3 exactly.

```
price = 99.95
pi_approx = 3.14159
small = 0.000001
```

Strings. Sequences of Unicode characters enclosed in single or double quotes. The two forms are equivalent; pick whichever requires less escaping for your content.

```
greeting = "Hello, world!"
name = 'Borty'
quote = "She said \"hi\" to me"
multiline = 'line one\nline two'
```

Inside a string literal, the backslash introduces an escape sequence. The supported escapes are **\n** (newline), **\t** (tab), **** (literal backslash), and **\"** or **\'** (matching the opening quote). Unrecognized escapes pass through unchanged.

Booleans. Exactly two values: **true** and **false**, written in lowercase. They are distinct from integers — unlike C, comparing true to 1 returns false.

```
done = true
ready = false
```

Nil. A single value meaning the absence of anything useful. Equivalent to Python's None or Lua's nil. Returned by functions that have nothing meaningful to return.

```
result = nil
if result == nil
    print("nothing to see here")
end
```

2.3 Variables and assignment

Variables are created by simple assignment with the equals sign. There is no separate declaration syntax; the first assignment creates the binding, and subsequent assignments update it. Names may contain letters, digits, and underscores, but must not start with a digit. They are case-sensitive, so **age** and **Age** are different variables.

```
score = 0
score = score + 10
score = score * 2
print(score)         # 20
```

Variables are strongly typed in the sense that each value carries its type at runtime, but the language is dynamically typed: the same name can hold values of different types over its lifetime. This flexibility is convenient for quick scripts but can hide bugs; the discipline of sticking to one type per variable is recommended for anything longer than a few lines.

```
x = 42           # x is an integer
x = "hello"      # x is now a string (legal but questionable)
x = [1, 2, 3]    # x is now a list (definitely questionable)
```

2.4 Arithmetic operators

BoRLang provides the five common arithmetic operators. They behave as you would expect from Python: integer division uses the forward-slash (returns a float if either operand is a float), and modulo handles negative numbers with Python semantics.

```
print(2 + 3)     # 5
print(10 - 4)    # 6
print(6 * 7)     # 42
print(10 / 3)    # 3.3333333333333335
print(10 % 3)    # 1
```

Operator precedence follows the standard mathematical ordering: multiplication and division bind tighter than addition and subtraction, and parentheses override everything. The modulo operator has the same precedence as multiplication.

```
print(2 + 3 * 4)     # 14 (not 20)
print((2 + 3) * 4)   # 20
print(10 - 2 - 3)    # 5 (left-associative)
```

2.5 Comparison operators

Six comparison operators produce boolean results: equality (**==**), inequality (**!=**), less-than (**<**), greater-than (**>**), and their non-strict variants (**<=** and **>=**). They work on numbers, strings (lexicographic order), and booleans.

```
print(5 > 3)          # true
print("apple" < "banana")  # true
print(3 == 3.0)       # true (numeric comparison)
print("3" == 3)       # false (different types)
```

2.6 Logical operators

The three logical operators are spelled as words, not symbols: **and**, **or**, and **not**. Like Python, they short-circuit — **and** stops evaluating as soon as it sees a false value, and **or** stops as soon as it sees a true one. The value they return is the last one they examined, which lets you use them for more than pure boolean logic.

```
print(true and false)       # false
print(true or false)        # true
print(not true)             # false

# Short-circuit for defaults:
name = user_name or "anonymous"
# If user_name is nil or false, name becomes "anonymous"
```

2.7 String concatenation

The plus sign doubles as string concatenation when either operand is a string. BoRLang is forgiving here: if one side is a number and the other is a string, the number is converted automatically. This differs from strict Python, where you would have to call **str()** explicitly.

```
greeting = "Hello"
name = "Borty"
print(greeting + ", " + name + "!")   # Hello, Borty!

age = 30
print("I am " + age + " years old")   # works!
print("I am " + str(age) + " years")  # also works
```

NOTE. Automatic number-to-string conversion only applies when one side is already a string. Writing **42 + 7** still gives the arithmetic sum 49, not the string "427".

2.8 Reserved keywords

The following identifiers are reserved by the language and cannot be used as variable names: **if, elif, else, end, while, for, in, def, return, true, false, nil, and, or, not**. Attempting to assign to one of these produces a parse error.

2.9 Semicolons and newlines

BoRLang treats newlines as statement separators, but not strictly. You can write multiple statements on one line by leaving a space between them (no semicolon is required or recognized). Long expressions may be split across lines freely because the parser tracks bracket nesting.

```
x = 1  y = 2  z = 3    # three statements, one line (legal)

result = some_function(
    first_arg,
    second_arg,
    third_arg
)   # line breaks inside parentheses are fine
```

3 Control Flow

Every non-trivial program makes decisions and repeats actions. BoRLang offers three control-flow constructs — the conditional **if** statement, the **while** loop, and the **for-in** loop. All three are terminated by the keyword **end**, borrowed from Lua and Ruby. There is no indentation-based block syntax as in Python; the **end** keyword is the only way to close a block.

3.1 The if statement

The simplest form of conditional takes a boolean expression and a body that runs only when the expression is true:

```
age = 20
if age >= 18
    print("You can vote")
end
```

An optional colon after the condition is accepted but not required; the interpreter silently consumes it if present. This is a concession to readers who find the Python-style colon easier on the eyes. Indentation inside the block is stylistic and ignored by the parser; a four-space indent is the convention.

To handle the false case, add an **else** branch:

```
age = 16
if age >= 18
    print("You can vote")
else
    print("Come back in " + (18 - age) + " years")
end
```

For chained conditions, use **elif**. You may have any number of **elif** branches; they are checked top to bottom and only the first matching one executes.

```
score = 75
if score >= 90
    grade = "A"
elif score >= 80
    grade = "B"
elif score >= 70
    grade = "C"
elif score >= 60
    grade = "D"
else
    grade = "F"
end
print("Grade: " + grade)
```

NOTE. Unlike C or Java, BoRLang does not allow a single-statement if without its body block. Every if must have at least a body and an end, even if the body is a single print.

3.2 Truthiness

BoRLang follows Python's rules for truthiness. A value is considered true in a conditional unless it is one of the following falsy values: **false**, **nil**, the integer **0**, the float **0.0**, the empty string **""**, the empty list **[]**, or the empty dictionary **{}**. Everything else is true.

```
items = []
if items
    print("we have items")
else
    print("list is empty")
end
# Output: list is empty
```

3.3 The while loop

A while loop repeats its body as long as its condition is true. The condition is checked before each iteration, so if it starts false the body never runs at all. The loop terminates when the condition becomes false; the programmer is responsible for ensuring this happens, because BoRLang does not yet have a **break** statement.

```
# Sum of numbers 1 through 10
total = 0
n = 1
while n <= 10
    total = total + n
    n = n + 1
end
print("Sum: " + total)   # Sum: 55
```

WARNING. It is easy to write an infinite loop by forgetting to advance the loop variable. The REPL catches Ctrl-C and returns you to the prompt; a file-mode script will hang until you kill it. Keep one eye on the termination condition when writing loops.

3.4 The for-in loop

The for loop iterates over a collection — a list, a string, a dictionary, or the result of **range()**. On each iteration, the loop variable is bound to the next element:

```
for word in ["red", "green", "blue"]
    print(word)
end
# red
# green
# blue
```

To iterate over a range of integers, pair the for loop with the **range(start, end)** built-in. The range is half-open, mirroring Python: **range(1, 5)** yields the integers 1, 2, 3, and 4 — not 5.

```
# Print squares of 1..5
for n in range(1, 6)
    print(n + " squared is " + (n * n))
end
```

Iterating over a dictionary yields its keys, in insertion order:

```
prices = {apple: 1.2, bread: 2.5, milk: 0.9}
for item in prices
```

```
    print(item + ": $" + prices[item])
end
```

3.5 Nesting loops and conditionals

Control-flow statements nest freely. Each **if**, **while**, or **for** must be closed by its own **end**. A common pattern is filtering items that meet a condition while iterating:

```
# Collect only positive numbers
numbers = [-3, 7, -1, 4, 0, 9, -2]
positive = []
for n in numbers
    if n > 0
        push(positive, n)
    end
end
print(positive)   # [7, 4, 9]
```

3.6 Missing features

BoRLang 3.1 does not yet provide **break**, **continue**, or a switch/match statement. These are planned for version 3.2. For the moment, you can emulate **break** with a flag variable and the loop condition:

```
# Emulating break: find first negative number
items = [3, 5, 7, -2, 8, -1]
found = false
i = 0
while i < len(items) and not found
    if items[i] < 0
        print("Found negative at index " + i)
        found = true
    end
    i = i + 1
end
```

4 Functions & Closures

Functions let you package a sequence of statements under a name and invoke them later with different inputs. BoRLang treats functions as first-class values: you can store them in variables, pass them as arguments, return them from other functions, and nest them arbitrarily. Nested functions automatically capture variables from their enclosing scope — this feature is called **lexical closures**, and it makes BoRLang more powerful than a typical teaching language.

4.1 Defining functions

A function definition starts with **def**, followed by the function name, a parameter list in parentheses, and a body terminated by **end**. The **return** keyword sends a value back to the caller.

```
def add(a, b)
    return a + b
end

result = add(3, 4)
print(result)   # 7
```

A function without an explicit return statement returns **nil**. This is useful for procedures that do work but have no meaningful output — printing to the screen, modifying a list in place, and so on.

```
def greet(name)
    print("Hello, " + name)
end

result = greet("Borty")
print(result)   # nil
```

4.2 Multiple return paths

A function may contain several return statements; the first one encountered during execution wins. This is often cleaner than a single return at the bottom with nested if/else.

```
def classify(n)
    if n < 0
        return "negative"
    end
    if n == 0
        return "zero"
    end
    return "positive"
end

print(classify(-5))   # negative
print(classify(0))    # zero
print(classify(42))   # positive
```

4.3 Scoping rules

Every function has its own local scope. Parameters are bound in this scope, as are any variables assigned inside the body for the first time. When the function returns, the local scope is destroyed and its variables cease to exist. Names defined outside the function remain visible inside it — BoRLang uses **lexical scoping**, meaning the rules are determined by the source-code structure, not the call order.

```
pi = 3.14159        # outer scope

def circle_area(r)
    return pi * r * r   # pi is visible from the outer scope
end

print(circle_area(10))
```

Assignment inside a function writes to the nearest scope that already has that name, walking up through nested scopes. If no enclosing scope has the name, a new binding is created in the current (innermost) scope. This rule keeps the language simple but means you should be careful when assigning to a name that happens to exist in an outer scope — you will modify it, not create a shadow.

4.4 Closures

A closure is a function that remembers the environment in which it was created, even after that environment has gone out of scope. In BoRLang, every function is automatically a closure.

```
def make_adder(x)
    def inner(y)
        return x + y
    end
    return inner
end

add5 = make_adder(5)
add10 = make_adder(10)

print(add5(3))     # 8
print(add10(3))    # 13
print(add5(100))   # 105
```

Notice that the inner function still has access to **x** after **make_adder** has returned. The parameter **x** is captured by reference in the closure's environment, and each call to **make_adder** creates a fresh environment — that is why **add5** and **add10** remember different values.

4.5 Stateful closures

A closure can modify the variables it captures. This lets you build small state machines without classes:

```
def make_counter(start)
    count = start
    def tick()
        count = count + 1
        return count
    end
```

```
    return tick
end

c = make_counter(0)
print(c())   # 1
print(c())   # 2
print(c())   # 3
```

The **count** variable lives in the closure environment of **tick**. Each call to **tick** increments the same variable. Calling **make_counter** a second time would create an independent counter with its own count.

4.6 Higher-order functions

Because functions are first-class values, you can pass them as arguments and return them as results. This enables functional-style programming patterns that are hard to express in languages without this feature.

```
def apply_twice(fn, x)
    return fn(fn(x))
end

def square(n)
    return n * n
end

print(apply_twice(square, 3))   # 81, because (3²)² = 81
```

4.7 Recursion

A function can call itself. This is called recursion and is useful for problems that decompose naturally into smaller instances of themselves. Factorial is the canonical example:

```
def factorial(n)
    if n <= 1
        return 1
    end
    return n * factorial(n - 1)
end

print(factorial(10))   # 3628800
```

WARNING. BoRLang does not optimize tail calls, so deeply recursive functions can overflow Python's call stack (typically after about one thousand frames). Convert to iterative form for large inputs.

4.8 Anonymous calls and pipelines

BoRLang does not currently have a lambda or fn expression syntax for anonymous functions. If you want an inline callable, define a named function and pass its name. This is slightly more verbose than Python but keeps the language smaller.

5 Collections

Real programs rarely manipulate individual values in isolation; they group them into collections. BoRLang provides three collection types: **lists** (ordered sequences), **dictionaries** (key-value mappings), and **strings** (sequences of characters treated as first-class values). This chapter shows how to create, access, and manipulate each of them.

5.1 Lists

A list is an ordered, mutable sequence of values. List literals use square brackets with comma-separated elements; the elements can be of any type and may be mixed.

```
empty = []
numbers = [1, 2, 3, 4, 5]
mixed = [1, "hello", true, 3.14, nil]
nested = [[1, 2], [3, 4], [5, 6]]
```

Access an element by its integer index. BoRLang uses zero-based indexing: the first element is at index 0, the second at 1, and so on. Negative indices count from the end, so **-1** is the last element. Out-of-range indices raise an error rather than silently returning nil.

```
fruit = ["apple", "banana", "cherry"]
print(fruit[0])    # apple
print(fruit[2])    # cherry
print(fruit[-1])   # cherry
```

Modify an element by assigning to its index. The list grows and shrinks with **push** (append to end) and **pop** (remove and return last element):

```
colors = ["red", "green", "blue"]
colors[1] = "GREEN"
print(colors)        # [red, GREEN, blue]

push(colors, "yellow")
print(colors)        # [red, GREEN, blue, yellow]

removed = pop(colors)
print(removed)       # yellow
print(colors)        # [red, GREEN, blue]
```

5.2 Dictionaries

A dictionary (sometimes called a table or map) stores values indexed by keys. Dictionary literals use curly braces, with keys separated from values by colons:

```
user = {name: "Borty", age: 30, city: "Milan"}
print(user["name"])   # Borty
```

Dictionary keys in BoRLang literals are written without quotes around the key name. This is a small syntactic sugar: the key is still a string internally. You use quotes when indexing:

```
# Literal: keys are bare words
config = {host: "localhost", port: 8080}
```

```
# Access: keys are quoted strings
print(config["host"])   # localhost
```

Add or update entries by assigning to a key:

```
user = {name: "Borty", age: 30}
user["email"] = "borty@example.com"
user["age"] = 31
print(user)
```

The **keys** and **values** built-ins return lists that you can iterate over:

```
prices = {apple: 1.2, bread: 2.5, milk: 0.9}

for k in keys(prices)
    print(k + ": $" + prices[k])
end

total = 0
for v in values(prices)
    total = total + v
end
print("Total: $" + total)
```

5.3 Strings in depth

Strings are technically immutable sequences of characters. You can index into them like lists, but you cannot assign to an index (any apparent mutation is really creation of a new string).

```
word = "BoRLang"
print(word[0])        # B
print(word[-1])       # g
print(len(word))      # 7
```

The standard library provides a rich set of string operations. Each returns a new string rather than modifying in place:

```
s = "  Hello BoRLang  "
print(upper(s))           # "  HELLO BORLANG  "
print(lower(s))           # "  hello borlang  "
print(trim(s))            # "Hello BoRLang"
print(replace(s, "BoRLang", "World"))  # "  Hello World  "
print(contains(s, "BoR"))  # true
print(startswith(s, "  He"))   # true
```

Split and join for parsing and assembly:

```
csv_line = "apple,banana,cherry,date"
items = split(csv_line, ",")
print(items)              # [apple, banana, cherry, date]
print(items[2])           # cherry

rejoined = join(items, " | ")
print(rejoined)           # apple | banana | cherry | date
```

5.4 Regular expressions

Three built-ins expose the Python regex engine for pattern-matching work: **match** returns the first match, **gmatch** returns all non-overlapping matches, and **gsub** performs substitutions.

```
text = "My phone is 555-1234 and backup is 555-9876"

# First match
first = match("[0-9]+-[0-9]+", text)
print(first)         # 555-1234

# All matches
all = gmatch("[0-9]+-[0-9]+", text)
print(all)           # [555-1234, 555-9876]

# Substitute
clean = gsub(text, "[0-9]+-[0-9]+", "[REDACTED]")
print(clean)         # My phone is [REDACTED] and backup is [REDACTED]
```

5.5 The len function

The **len** built-in returns the length of any collection: the number of elements in a list, the number of key-value pairs in a dictionary, or the number of characters in a string. It returns 0 for empty collections and raises an error for types that have no defined length.

```
print(len([1, 2, 3]))             # 3
print(len("BoRLang"))             # 7
print(len({a: 1, b: 2, c: 3}))    # 3
print(len([]))                    # 0
```

6 Standard Library Reference

This chapter is a reference, not a tutorial. It lists every built-in function available in BoRLang 3.1, grouped by module, with a short description of the inputs and outputs. Use it when you are writing code and need to recall exact names or argument orders. For narrative explanations of how to use these functions for real work, see chapters 7 through 10.

In the listings below, each function is shown with its signature and a brief description. Arguments marked with a question mark are optional.

6.1 I/O and conversion

```
print(... args)              Print args separated by spaces + newline
println(... args)            Alias for print
len(collection)              Number of elements
str(value)                   Coerce any value to string
int(value)                   Coerce to integer (truncates floats)
float(value)                 Coerce to float
bool(value)                  Coerce to boolean by truthiness rule
typeof(value)                Type name: "int", "string", etc.
```

6.2 Strings

```
upper(s)                     Uppercase copy
lower(s)                     Lowercase copy
trim(s)                      Strip leading and trailing whitespace
split(s, delim)              Split on delimiter, return list
join(list, delim)            Join with delimiter, return string
replace(s, old, new)         Replace all occurrences of old
startswith(s, prefix)        Test prefix
endswith(s, suffix)          Test suffix
contains(s, sub)             Test substring presence
sub(s, start, end?)          Slice: s[start..end]
match(pattern, text)         First regex match, or nil
gmatch(pattern, text)        All non-overlapping regex matches
gsub(text, pattern, repl)    Regex substitution
```

6.3 Math

```
abs(n)                      Absolute value
floor(n)                    Round down to int
ceil(n)                     Round up to int
round(n, decimals?)         Round to N decimal places
sqrt(n)                     Square root
pow(base, exp)              Exponentiation
log(n, base?)               Natural or base-N logarithm
log10(n)                    Base-10 logarithm
sin(n) / cos(n) / tan(n)    Trigonometry (radians)
min(... args)               Minimum of arguments or list
max(... args)               Maximum of arguments or list
sum(list)                   Sum of numeric list
random(a?, b?)              Random integer in [a, b]
range(start, end, step?)    Integer range as list
pi() / e()                  Mathematical constants
```

6.4 Collections

```
push(list, item)            Append item (mutates list)
pop(list)                   Remove and return last element
sort(list)                  Return sorted copy
reverse(list)               Return reversed copy
keys(dict)                  List of dictionary keys
values(dict)                List of dictionary values
table(... pairs)            Make dict from alternating k, v
```

6.5 JSON

```
json_encode(value)          Serialize any value to JSON string
json_decode(text)           Parse JSON string, return value
```

6.6 File I/O

```
read(path)                  Read entire file as string, or nil
write(path, text)           Write text, overwriting file
append(path, text)          Append text to file
ls(path?)                   List directory entries
exists(path)                File or directory exists?
mkdir(path)                 Create directory (including parents)
rm(path)                    Delete file or empty directory
```

6.7 System and time

```
time()                      Unix timestamp (seconds since epoch)
date()                      ISO date "YYYY-MM-DD"
datetime()                  ISO timestamp
sleep(seconds)              Pause execution
env(name, default?)         Read environment variable
cwd()                       Current working directory
```

6.8 Crypto and encoding

```
md5(text)                    MD5 hash, hex string
sha1(text)                   SHA-1 hash
sha256(text)                 SHA-256 hash
sha512(text)                 SHA-512 hash
base64_encode(text)          Base64 encoding
base64_decode(text)          Base64 decoding
hex_encode(text)             Hex encoding
hex_decode(text)             Hex decoding
url_encode(text)             URL percent-encoding
url_decode(text)             URL decoding
uuid()                       Random 128-bit UUID
token(length?)               Random URL-safe token
gen_password(length?)        Random password
```

6.9 Machine learning (chapter 7)

```
linear_regression(x, y)      Fit y = slope*x + intercept
kmeans(data, k)              K-means clustering
pca(data, n_components)      Principal component analysis
normalize(data)              L2-normalize rows
sigmoid(x) / relu(x)         Activation functions
softmax(x)                   Softmax probability distribution
mean(x) / std(x)             Statistics
dot(a, b) / matmul(a, b)     Linear algebra
similarity(a, b)             Cosine similarity
```

6.10 Vector store (chapter 8)

```
vector_create(dim?)          Create empty vector store
vector_add(store, text)      Embed and index text
vector_search(store, q, k?)  Top-k nearest matches
```

6.11 Memory (chapter 9)

```
memory_new(path?)            Create SQLite-backed memory
memory_save(mem, key, value) Persist a value
memory_recall(mem, key)      Retrieve a value, or nil
memory_forget(mem, key)      Delete a value
memory_stats(mem)            Return {entries, size_bytes}
```

6.12 Quantum-inspired (chapter 10)

```
quantum_encode(data)         Amplitude encoding
quantum_decode(encoded)      Inverse of quantum_encode
quantum_measure(state)       Sample index from |amp|² distribution
svd_compress(matrix, rank)   Truncated SVD
compress_zh(text)            Dictionary coding with CJK symbols
decompress_zh(payload)       Inverse of compress_zh
compress_zlib(text)          Real byte-level zlib compression
decompress_zlib(payload)     Inverse of compress_zlib
```

6.13 BoRFlow (chapter 11)

```
flow_new(name)                Create empty flow
flow_add(f, id, type, cfg)    Add a node
flow_connect(f, from, to)     Add default edge
flow_branch(f, cond, t, f)    Connect condition branches
flow_set_start(f, id)         Override the start node
flow_run(f, initial?)         Execute the flow
flow_trace(f)                 Get last execution log
flow_to_json(f)               Serialize flow structure
```

7 Machine Learning Module

BoRLang ships with a small, practical selection of machine-learning primitives. They cover supervised regression, unsupervised clustering, dimensionality reduction, normalization, and the usual activation functions. Behind the scenes they wrap NumPy and scikit-learn, so their numerical results are identical to what you would get writing Python directly. The advantage of having them in BoRLang is that you can combine them with the rest of the language — control flow, closures, and eventually BoRFlow blocks — without context-switching between syntaxes.

7.1 Linear regression

The **linear_regression** function fits a line of the form y = slope × x + intercept through a set of paired data points. It takes two lists: the x-values and the y-values. The returned model is a structure with three attributes — **slope**, **intercept**, and **r2** (the coefficient of determination) — and a method **predict(x)** that produces y-values for new x-values.

```
# Data: house size (sqm) vs price (thousand euros)
sqm = [50, 65, 80, 95, 110, 125, 140]
price = [125, 165, 210, 245, 295, 340, 380]

model = linear_regression(sqm, price)
print("slope     = " + round(model.slope, 3))
print("intercept = " + round(model.intercept, 3))
print("R²        = " + round(model.r2, 4))

# Predict price for 100 sqm
predicted = model.predict([100])
print("Predicted price for 100sqm: " + predicted[0])
```

The coefficient of determination (R^2) tells you how well the line fits the data. A value of 1.0 means perfect fit; 0.0 means the line is no better than predicting the average y-value. In practice, real-world data rarely exceeds 0.9 without overfitting.

> **NOTE.** Linear regression assumes a linear relationship. If your data is curved, a simple line will produce poor predictions even with high R^2. Always look at a scatter plot before drawing conclusions.

7.2 K-means clustering

Clustering is an unsupervised task: given a set of data points, group them into categories without being told the correct labels. The k-means algorithm partitions points into k clusters by iteratively moving cluster centers toward the points nearest to them.

```
# 2-D points that form two obvious groups
data = [[1, 1], [1, 2], [2, 1], [2, 2],
        [10, 10], [10, 11], [11, 10], [11, 11]]

result = kmeans(data, 2)
print("labels  = " + result["labels"])
print("centers = " + result["centers"])
print("inertia = " + result["inertia"])
```

The returned dictionary contains three fields. **labels** is a list of integers — one per input point — assigning each point to a cluster index. **centers** is a list of cluster center coordinates. **inertia** is the sum of squared distances of every point to its assigned center, a rough measure of how tight the clusters are.

Choosing k is a problem on its own. A common heuristic is the **elbow method**: run k-means for k = 1, 2, 3, ..., plot the inertia, and pick the k where the curve bends sharply. BoRLang makes this easy because you can express the whole sweep as a for-loop:

```
inertias = []
for k in range(1, 10)
    r = kmeans(data, k)
    push(inertias, r["inertia"])
    print("k=" + k + "  inertia=" + round(r["inertia"], 2))
end
```

7.3 Principal component analysis

When data has many features, some of them are redundant — they are correlated with other features and add little information. PCA finds a smaller set of new features (called components) that together capture as much of the variance as possible. It is used for visualization (reducing to 2-D or 3-D so humans can see the data) and as a preprocessing step before other algorithms.

```
# 5-dimensional data, reduce to 2 components
data = [
    [1, 2, 3, 4, 5],
    [2, 4, 6, 8, 10],
    [5, 4, 3, 2, 1],
    [1, 1, 1, 1, 1]
]
result = pca(data, 2)
print("reduced = " + result["reduced"])
print("explained variance = " + result["explained_variance_ratio"])
```

The **explained_variance_ratio** tells you what fraction of the total variation each component accounts for. If the first two components together explain 95% of the variance, reducing from 50 dimensions to 2 has lost very little information.

7.4 Normalization

Many ML algorithms work best when features are on similar scales. L2 normalization scales each row so that its Euclidean length is 1. This is particularly important for distance-based methods like k-means and vector search.

```
# Three 3-D points with wildly different magnitudes
data = [
    [100, 200, 300],
    [1, 2, 3],
    [0.01, 0.02, 0.03]
]
normalized = normalize(data)
print(normalized)
# All three rows now have length 1 and point in the same direction
```

7.5 Activation functions

Three non-linear activation functions are provided. All accept either a scalar or an array and apply the function elementwise. They are building blocks for manually-implementing simple neural networks, but mostly they are useful as numerical utilities.

```
x = [-2, -1, 0, 1, 2]

print(sigmoid(x))     # Maps R to (0, 1); sigmoid(0) = 0.5
print(relu(x))        # max(0, x); zeros out negatives
print(softmax(x))     # Probability distribution summing to 1
```

Softmax is particularly useful for converting arbitrary scores into probabilities: it exponentiates each value and then normalizes so the whole output sums to 1. Commonly used at the output layer of classifiers.

7.6 Statistical aggregates

For quick data inspection without loading the full numpy namespace, BoRLang exposes **mean**, **std**, **sum**, **dot**, and **matmul** as simple built-ins.

```
scores = [82, 76, 95, 88, 91, 74, 89]
print("mean  = " + round(mean(scores), 2))
print("std   = " + round(std(scores), 2))
print("sum   = " + sum(scores))

# Dot product and matrix multiplication
a = [1, 2, 3]
b = [4, 5, 6]
print("dot   = " + dot(a, b))   # 1*4 + 2*5 + 3*6 = 32
```

7.7 Cosine similarity

The **similarity** function computes the cosine of the angle between two vectors. It returns 1 when the vectors point in exactly the same direction, 0 when they are perpendicular, and -1 when opposite. Widely used in information retrieval to compare documents represented as bag-of-words vectors.

```
v1 = [1, 2, 3, 4]
v2 = [2, 4, 6, 8]    # exactly parallel to v1
v3 = [-1, -2, -3, -4]  # exactly anti-parallel

print(similarity(v1, v2))    # 1.0
print(similarity(v1, v3))    # -1.0
```

7.8 Putting it together: a tiny classifier

The following example trains a line to separate two clusters of points, then uses the line to classify a new point. It touches regression, prediction, and the broader workflow of a supervised ML task.

```
# Training data: x is a feature, y is the target (0 or 1)
x = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
y = [0, 0, 0, 0, 0, 1, 1, 1, 1, 1]

# Fit a line — not a proper classifier but illustrative
model = linear_regression(x, y)
```

```
# Decision rule: if predicted value > 0.5, class = 1
def classify(value)
    p = model.predict([value])[0]
    if p > 0.5
        return 1
    end
    return 0
end

print(classify(3))    # 0
print(classify(7))    # 1
print(classify(5.5))  # around the decision boundary
```

7.9 Worked example: customer segmentation

To pull the ML primitives together into something realistic, let us walk through a customer-segmentation exercise. The scenario: you have exported purchase data for a hundred customers, each described by three numbers — average order value, number of orders, and days since last order. You want to group the customers into a few segments so the marketing team can target each segment with a different campaign.

The workflow has three stages. First, normalize the features so that order value (which can be hundreds of euros) does not dominate days-since-purchase (which is a number between zero and perhaps 365). Second, run k-means to find cluster centers. Third, label each cluster by interpreting its center values.

```
# Sample data: 6 customers, 3 features each
customers = [
    [120, 30, 5],    # high-value regular
    [110, 28, 3],    # high-value regular
    [35, 5, 180],    # lapsed
    [40, 4, 160],    # lapsed
    [200, 2, 10],    # recent one-time buyer
    [180, 2, 15]     # recent one-time buyer
]

# 1. Normalize so features have comparable scales
normalized = normalize(customers)

# 2. Cluster
result = kmeans(normalized, 3)
print("Assignments: " + result["labels"])
print("Inertia:     " + round(result["inertia"], 3))

# 3. Inspect each cluster center
for cluster_idx in range(0, 3)
    center = result["centers"][cluster_idx]
    print("Cluster " + cluster_idx + " center: " + center)
    # In a real dashboard you would show mean feature values
    # for the unnormalized data here
end
```

The output tells you which customers fall into each segment. Interpreting the centers back into business language is where a human still beats the algorithm: a cluster whose center has high order value and low days-since might be labeled "loyal big spenders"; a cluster with low order value and high days-since might be "lapsed casuals". The same k-means algorithm serves as the technical backbone of customer relationship management systems sold for thousands of euros; the BoRLang

version is fifteen lines of code.

7.10 Combining ML with flows

The real power of having ML built into BoRLang is that ML operations can appear as blocks within a BoRFlow pipeline. You can retrieve documents from an API, embed them, cluster them, and route to different LLM prompts based on cluster — all within a single declarative flow. Here is a sketch:

```
# Classify incoming tickets into technical vs non-technical
# using k-means on example-embeddings, then route to different prompts

def classify_handler(config, ctx)
    text = ctx["ticket"]
    vec = simple_embed(text)
    # Precomputed cluster centers representing technical/general
    tech_dist = euclidean(vec, TECH_CENTER)
    gen_dist = euclidean(vec, GEN_CENTER)
    if tech_dist < gen_dist
        return "technical"
    end
    return "general"
end

f = flow_new("triage")
flow_register(f, "classify", classify_handler)
flow_add(f, "route", "action", {name: "classify"})
flow_add(f, "is_tech", "condition", {
    left: "{{route}}", op: "==", right: "technical"
})
# ... and so on
```

This pattern — ML classification followed by rule-based routing — handles a huge class of real problems. Customer support routing, email filtering, document triage, and content moderation all fit this mold. The ML does what ML is good at (fuzzy pattern recognition); the flow does what flows are good at (explicit sequencing). Each stays focused on its strength.

8 Vector Store & Semantic Search

Traditional search uses keyword matching: a document matches a query if the query's words appear in it. This misses synonyms, paraphrases, and related concepts. Vector search solves this by converting each document and each query into a high-dimensional vector, then ranking by the angle between them. Documents with similar meanings produce vectors that point in similar directions, so they score high even if they share no words.

8.1 The vector store abstraction

A vector store is three things bundled together: an embedder (converts text to vector), an index (stores the vectors), and a search function (finds nearest neighbors of a query). BoRLang exposes all three through a single object created with **vector_create**.

```
store = vector_create(128)      # 128-dimensional embeddings
```

By default, the store uses a **hash embedding**: each word in the text is hashed to a bucket in the vector. This is fast, deterministic, and requires no external model — but it is not truly semantic. The words "car" and "automobile" will hash to different buckets and therefore appear unrelated. For real semantic search, you can upgrade to a sentence-transformer model by installing the **sentence-transformers** package and calling **store.load_embedder()**.

8.2 Indexing documents

Add documents one at a time with **vector_add**. Each document is embedded and stored with an auto-assigned integer ID.

```
store = vector_create(256)

docs = [
    "The sun is a star at the center of our solar system",
    "Python is a popular programming language for data science",
    "Machine learning models need large amounts of training data",
    "Pizza originated in Naples, Italy in the 18th century",
    "The moon orbits the Earth every 27.3 days"
]

for d in docs
    vector_add(store, d)
end
print("Indexed " + len(docs) + " documents")
```

8.3 Searching the store

Run a query with **vector_search**. It takes the store, the query string, and an optional **k** (number of results to return; default 5). It returns a list of hits, each a dictionary with **id**, **score**, and **text**.

```
hits = vector_search(store, "astronomy and planets", 3)

print("Top 3 results:")
```

```
for h in hits
    print("  score=" + round(h["score"], 3))
    print("         " + h["text"])
end
```

The score is a similarity measure in the range [-1, 1], where 1 is a perfect match and 0 is unrelated. With the hash embedder, scores are typically small positive numbers because the bucket overlap between unrelated texts is low; with a semantic embedder, scores of 0.7 or higher for related documents are common.

8.4 Building a minimal RAG pipeline

Retrieval-augmented generation (RAG) is a pattern where a language model answers questions using documents retrieved by vector search. The workflow is always the same: index the documents, search on the user's question, feed the top hits to the model as context. Here is the retrieval half of a RAG pipeline in pure BoRLang:

```
# 1. Index your knowledge base
kb = vector_create(256)
articles = read("knowledge_base.txt")
for paragraph in split(articles, "\n\n")
    if len(paragraph) > 50
        vector_add(kb, paragraph)
    end
end

# 2. Retrieve on the user's question
def answer(question)
    hits = vector_search(kb, question, 3)
    context = ""
    for h in hits
        context = context + h["text"] + "\n---\n"
    end
    return context
end

print(answer("How do neural networks learn?"))
```

The generation half — feeding this context to a language model and getting an answer — is the subject of chapter 11 on BoRFlow.

8.5 FAISS backend

By default, the vector store does brute-force search: it computes the similarity between the query and every indexed document. This is fine for a few thousand documents but becomes slow for millions. Install **faiss-cpu** to upgrade to FAISS, Meta's industry-standard similarity-search library, which uses approximate algorithms to search billions of vectors in milliseconds.

```
# The store automatically detects FAISS; no code changes required.
# Just pip install faiss-cpu.
```

8.6 Semantic embeddings

True semantic search requires learning embeddings from a language model. The **sentence-transformers** library provides pretrained models that map sentences to meaningful vectors. BoRLang's store automatically picks up the library if it is installed; otherwise it falls back to the hash embedder.

```
# Automatic upgrade when sentence-transformers is present.
# No BoRLang code changes needed.
# pip install sentence-transformers
```

9 Memory Sandbox

A scripting language without persistence is limited: you can compute, but you cannot remember anything between runs. BoRLang's memory module provides a simple key-value store backed by SQLite. It is not a full database — no indexes, no queries, no SQL — just a way to save values and get them back later. This is deliberately minimal; for anything more complex, drop to Python and use SQLAlchemy.

9.1 Creating a memory store

Call **memory_new** with a file path, or with no arguments for an in-memory store that vanishes when the program exits. In-memory mode is useful for tests and short-lived scripts; file-backed mode persists across runs.

```
# Persistent
mem = memory_new("data.db")

# In-memory (volatile)
temp = memory_new()
```

9.2 Saving and recalling

Every memory entry is a key-value pair. Keys are strings; values can be any JSON-serializable BoRLang value — numbers, strings, booleans, lists, or dictionaries.

```
mem = memory_new("data.db")

# Store some things
memory_save(mem, "user_name", "Borty")
memory_save(mem, "last_run", datetime())
memory_save(mem, "preferences", {theme: "dark", language: "it"})

# Retrieve them
name = memory_recall(mem, "user_name")
prefs = memory_recall(mem, "preferences")
print("Welcome back, " + name)
print("Theme: " + prefs["theme"])
```

Saving to a key that already exists overwrites the previous value. If you want different versions, include a timestamp or counter in the key. Recalling a non-existent key returns **nil**, so you can check for first-run conditions:

```
count = memory_recall(mem, "run_count")
if count == nil
    count = 0
end
count = count + 1
memory_save(mem, "run_count", count)
print("This is run number " + count)
```

9.3 Forgetting

Remove a single entry with **memory_forget**. The function returns true if the key existed and was removed, false if it was not there to begin with.

```
if memory_forget(mem, "old_data")
    print("Removed")
else
    print("Nothing to remove")
end
```

9.4 Introspection

The **memory_stats** function returns a snapshot of the store: how many entries it has and how many bytes they occupy. Useful for debugging and for deciding when to clean up.

```
s = memory_stats(mem)
print("Entries: " + s["entries"])
print("Size:    " + s["size_bytes"] + " bytes")
```

9.5 A conversational agent's memory

One of the most common uses of persistent memory in an AI application is remembering the conversation history. Each exchange is saved with a timestamped key so the whole history can be reconstructed later.

```
mem = memory_new("chat_history.db")

def log_turn(role, text)
    ts = time()
    key = "turn_" + ts + "_" + role
    memory_save(mem, key, {role: role, text: text, time: ts})
end

# Log two exchanges
log_turn("user", "Hello")
log_turn("assistant", "Hi there! How can I help?")
log_turn("user", "What is BoRLang?")
log_turn("assistant", "A scripting language for ML and agents.")

print("Saved " + memory_stats(mem)["entries"] + " turns")
```

9.6 When to use and not use memory

The memory sandbox is ideal for configuration, user preferences, conversation history, and caching expensive computations. It is not designed for:

- large blobs — save files to disk and keep only the path in memory
- concurrent writes — SQLite's write lock means only one process can modify the store at once
- complex queries — there is no filter or sort operation, just key-lookup
- anything security-sensitive — entries are stored in plain JSON and the file is unencrypted

10 Quantum-Inspired & Compression

This chapter covers four functions that sit somewhat apart from the rest of the standard library: two quantum-inspired numerical transforms and two text-compression primitives. Their names connect to the original research roots of the BoRLang project, but they are classical algorithms — no actual quantum computer is involved. The chapter begins with a clear statement of what each does and does not do, to prevent the misunderstandings that plague the broader ecosystem of quantum-inspired tooling.

10.1 Amplitude encoding

In quantum computing, an n-qubit system can represent a superposition of 2^n classical states, each weighted by a complex **amplitude**. Measuring the system collapses it to one state with probability equal to the squared magnitude of the amplitude. This section simulates the mathematical structure — unit-norm vectors and probability-weighted sampling — on a classical computer. Nothing is entangled, there are no qubits, and no speedup over classical algorithms is claimed.

quantum_encode takes a list of numbers and returns a dictionary with a unit-norm version (the "state") and the original norm. The state has the property that the sum of squares of its elements equals 1.

```
signal = [3, 4, 0, 0]
enc = quantum_encode(signal)
print(enc["state"])   # [0.6, 0.8, 0.0, 0.0]
print(enc["norm"])    # 5.0  (sqrt(9+16))

# Verify unit norm
total = 0
for x in enc["state"]
    total = total + x * x
end
print(total)           # 1.0
```

quantum_decode inverts the operation, multiplying the state back by its stored norm to recover the original vector (up to floating-point rounding).

10.2 Measurement

quantum_measure samples an index from the state using the Born rule: each index i is drawn with probability equal to the square of the i-th amplitude. This is useful for weighted random sampling where the weights come from a vector of scores.

```
# State where index 2 has 80% weight
state = [0.2, 0.2, 0.8, 0.2]
# Sample many times and count outcomes
counts = {0: 0, 1: 0, 2: 0, 3: 0}
for i in range(0, 1000)
    idx = quantum_measure(state)
    counts[idx] = counts[idx] + 1
end
```

```
print(counts)   # Index 2 dominates, as expected
```

10.3 Truncated SVD compression

Singular value decomposition (SVD) factors any matrix into three parts: U × Σ × V^T, where Σ is a diagonal matrix of **singular values** sorted by importance. If most singular values are small, you can approximate the original matrix by keeping only the top few. This is how image and audio compression often works under the hood.

```
# A rank-1 matrix (each row is a multiple of the first)
matrix = [[1, 2, 3, 4], [2, 4, 6, 8], [3, 6, 9, 12]]

result = svd_compress(matrix, 1)   # keep only the top component
print("rank                = " + result["rank"])
print("compression ratio   = " + result["ratio"])
print("reconstruction err  = " + result["frobenius_error"])
```

For a rank-1 matrix, the rank-1 approximation is exact and the error is zero. For real data, the error grows as you keep fewer components, but not linearly — the first few components usually capture most of the information.

10.4 Dictionary coding (compress_zh)

The **compress_zh** function implements a visual compression scheme: frequent tokens in the input text are each replaced by a single character drawn from the CJK Unified Ideographs block. The output contains fewer characters — often 30 to 40 percent fewer — but more bytes when encoded as UTF-8, because each CJK character is three bytes versus one byte for ASCII.

> **WARNING.** This function is useful for visual compression (shortening text displayed to humans) but NOT for byte-level compression. The returned dictionary shows both metrics: **char_reduction** is usually positive, **byte_reduction_utf8** is usually negative. For actual byte savings, use **compress_zlib** instead.

```
text = "the quick brown fox jumps over the lazy dog " +
       "the quick brown fox runs fast the fox sleeps"
result = compress_zh(text)
print("compressed:        " + result["compressed"])
print("char reduction:    " + round(result["char_reduction"], 3))
print("byte reduction:    " + round(result["byte_reduction_utf8"], 3))
print("note:              " + result["note"])

# Roundtrip
back = decompress_zh(result)
print(back == text)   # true
```

10.5 Real compression with zlib

When you need actual byte-level compression, use **compress_zlib** — it wraps Python's stdlib zlib library and base64-encodes the result so it fits into a BoRLang string. The reduction is honest: on the same text above, zlib typically saves 20 to 30 percent of bytes.

```
text = "the quick brown fox jumps over the lazy dog " * 20
z = compress_zlib(text)
print("original bytes:   " + z["original_bytes"])
print("compressed bytes: " + z["compressed_bytes"])
print("byte reduction:   " + round(z["byte_reduction"], 3))
```

```
# Roundtrip
back = decompress_zlib(z)
print(back == text)   # true
```

11 BoRFlow: Agentic Workflows

BoRFlow is the part of the project that turns BoRLang from a scripting language into an agent-building platform. It is a small workflow engine modeled after n8n and RAGFlow: you declare blocks (nodes) and connect them with edges, then run the whole thing. State flows through a shared context dictionary that every block can read and write. Unlike visual-only tools, BoRFlow flows are defined in code — you get version control, diff-able workflows, and the full power of BoRLang to generate flows dynamically.

11.1 The mental model

A flow is a directed graph. Each vertex is a **block** — a small unit of work with inputs, a type, and a configuration dictionary. Each edge is a **transition** — a rule saying which block to run next. When you execute a flow, the engine starts at a designated node, runs its handler, stores the result in the context under the block's ID, and follows the appropriate edge to the next block.

There are three kinds of edges. The most common is the **default edge**, used for linear sequences: node A always leads to node B. A **condition block** has two outgoing edges, **on_true** and **on_false**, and the engine picks one based on the block's result. A **loop block** repeats a sub-sequence of nodes for each element of an iterable, then follows its single default edge.

11.2 Building your first flow

A minimal flow with three sequential print blocks:

```
f = flow_new("hello_flow")

flow_add(f, "greet",  "print", {message: "Hello!"})
flow_add(f, "middle", "print", {message: "Working..."})
flow_add(f, "done",   "print", {message: "Goodbye!"})

flow_connect(f, "greet",  "middle")
flow_connect(f, "middle", "done")

flow_run(f)
```

The first block added becomes the start node automatically. Use **flow_set_start** to override this if needed. Running the flow prints the three messages in order.

11.3 Block types

BoRFlow ships with eight built-in block types. Each expects a specific config dictionary and returns a specific kind of value.

print — Emit a message (templated with *{{key}}* placeholders from context). Returns the rendered message.

http — Perform an HTTP GET or POST via urllib. Config: *url*, *method*, *headers*, *body*, *timeout*. Returns *{status, body}*.

llm — Call a local Ollama endpoint. Config: *model*, *prompt*, *host* (default *localhost:11434*), *system*. Returns the LLM's response text.

transform — Extract a nested field via dotted path, or apply an operation (upper, lower, length, json_parse, json_stringify) to a context value.

condition — Compare two values with an operator (==, !=, <, >, <=, >=, contains, exists). Routes to *on_true* or *on_false*.

loop — Iterate a sub-sequence over a collection. Config: *over* (context key), *body* (list of node IDs), *item_key* (default "item").

python — Execute a user-provided Python callable against the context. Useful as an escape hatch.

action — Dispatch to a handler registered on the flow with *flow_register*. Used to extend BoRFlow with custom block types without modifying the library.

11.4 Context and templating

When a flow runs, the engine maintains a context dictionary. Initial values come from the second argument to **flow_run**. Each block's return value is stored in the context under the block's ID, so downstream blocks can reference it.

```
f = flow_new("demo")
flow_add(f, "hi", "print", {
    message: "Hello {{name}}, you are {{age}}!"
})
flow_run(f, {name: "Borty", age: 30})
# -> Hello Borty, you are 30!
```

Inside any string config value, the double-brace syntax **{{key}}** gets replaced at runtime with the value of that key in the context. Dotted paths work too: **{{user.name}}** reaches into a nested dictionary.

11.5 Branching with conditions

A condition block evaluates a comparison and directs the flow to one of two successor nodes based on the result. Use **flow_branch** to attach the true/false edges.

```
f = flow_new("age_check")

flow_add(f, "check", "condition", {
    left: "{{age}}", op: ">=", right: 18
})
flow_add(f, "adult", "print", {message: "You can vote"})
flow_add(f, "minor", "print", {message: "Too young to vote"})

flow_branch(f, "check", "adult", "minor")

flow_run(f, {age: 20})   # -> You can vote
flow_run(f, {age: 15})   # -> Too young to vote
```

11.6 Looping

A loop block iterates over a collection in the context, running its **body** nodes once per item. The current item is available in the context under the key named by **item_key** (default "item"). The loop block's result is a dictionary with **iterations** (count) and **results** (list of per-iteration contexts).

```
f = flow_new("batch")

flow_add(f, "upper", "transform", {
    input: "item", operation: "upper"
})
flow_add(f, "process", "loop", {
    over: "names", body: ["upper"]
})

flow_set_start(f, "process")
result = flow_run(f, {names: ["alice", "bob", "carol"]})

for r in result["process"]["results"]
    print(r["upper"])
end
# ALICE
# BOB
# CAROL
```

11.7 Calling LLMs

The **llm** block calls a locally-running Ollama server. Before using it, install Ollama (*https://ollama.ai*) and pull a model:

```
$ ollama pull qwen2.5:14b
```

Then from BoRLang:

```
f = flow_new("summarize")
flow_add(f, "ask", "llm", {
    model: "qwen2.5:14b",
    prompt: "Summarize in one sentence: {{text}}"
})

result = flow_run(f, {
    text: "BoRLang is a scripting language designed for ML..."
})
print(result["ask"]["response"])
```

NOTE. If Ollama is not running, the llm block raises a clear error rather than returning a fake completion. You can catch it with a condition block that checks the context for error markers.

11.8 HTTP requests

For real-world agents that call APIs, the **http** block is the workhorse. It supports all standard methods, custom headers, JSON bodies, and timeouts.

```
f = flow_new("api_call")
flow_add(f, "fetch", "http", {
    url: "https://jsonplaceholder.typicode.com/posts/1",
    method: "GET"
})
```

```
flow_add(f, "extract", "transform", {
    field: "fetch.body.title"
})
flow_connect(f, "fetch", "extract")

ctx = flow_run(f)
print(ctx["extract"])   # Title of the fetched post
```

11.9 Execution trace

After running a flow, call **flow_trace** to get a chronological log of every block that executed. Each entry records the block ID, type, status (ok or error), duration in milliseconds, and a truncated preview of the result.

```
flow_run(f, ctx)
for step in flow_trace(f)
    print(step["node"] + " [" + step["type"] + "] " +
          step["status"] + " (" + step["duration_ms"] + "ms)")
end
```

11.10 Serializing flows

A flow's structure — nodes, configs, edges — can be serialized to JSON with **flow_to_json**. This makes flows portable: save them to disk, send them over the network, or render them graphically in the companion **flow_designer.html** tool.

```
json = flow_to_json(f)
write("my_agent.bflow.json", json)
```

11.11 Custom action handlers

When none of the built-in block types match what you need, register a custom handler with **flow_register**. Handlers receive the block's config and the current context, and return the block's result.

```
def send_email(config, ctx)
    # ... real email-sending code goes here ...
    return {sent: true, to: config["to"]}
end

f = flow_new("notify")
flow_register(f, "email", send_email)

flow_add(f, "notify", "action", {
    name: "email", to: "admin@example.com"
})
flow_run(f)
```

Custom handlers let you wrap any Python library — Slack clients, SMTP, databases, message queues — as a block type. The agent code stays clean BoRLang while the integration details live in Python where they belong.

12 Cookbook: Real-World Recipes

The final chapter of this manual collects complete, ready-to-run recipes that solve common real-world problems. Each recipe is self-contained: copy it into a **.bor** file and it will work (assuming the appropriate dependencies). Use them as starting points for your own agents.

12.1 Retrieval-augmented question answering

Read a text file, split it into paragraphs, index them in a vector store, and answer questions by retrieving the three most relevant passages. This is the retrieval half of a RAG system — pair it with an **llm** block to get the full pipeline.

```
# rag_qa.bor
corpus = read("knowledge.txt")
store = vector_create(256)

for paragraph in split(corpus, "\n\n")
    if len(trim(paragraph)) > 50
        vector_add(store, trim(paragraph))
    end
end

def ask(question)
    print("Q: " + question)
    hits = vector_search(store, question, 3)
    print("Top 3 passages:")
    for h in hits
        print("  [" + round(h["score"], 3) + "] " + h["text"])
    end
end

ask("What is neural network?")
ask("How does backpropagation work?")
```

12.2 Daily data pipeline

Fetch data from a JSON API every run, extract a subset, and save it to disk with a dated filename. This is the skeleton of a cron-ready data collector.

```
# daily_fetch.bor
agent = flow_new("daily")

flow_add(agent, "fetch", "http", {
    url: "https://api.example.com/data",
    method: "GET"
})
flow_add(agent, "extract", "transform", {
    field: "fetch.body.results"
})
flow_connect(agent, "fetch", "extract")

ctx = flow_run(agent)
```

```
items = ctx["extract"]

filename = "data_" + date() + ".json"
write(filename, json_encode(items))
print("Saved " + len(items) + " items to " + filename)
```

12.3 Sentiment-gated support triage

Read a list of support tickets, classify each with an LLM, and route negative sentiment to a "high priority" bucket. Shows how conditions and loops combine in a realistic agent.

```
# triage.bor
tickets = [
    "Your product is amazing, thanks!",
    "Still broken after three emails.",
    "How do I reset my password?"
]

high = []
normal = []

agent = flow_new("classify")
flow_add(agent, "ask", "llm", {
    model: "qwen2.5:14b",
    prompt: "One word only: positive, neutral, or negative: {{text}}"
})

for t in tickets
    ctx = flow_run(agent, {text: t})
    s = lower(trim(ctx["ask"]["response"]))
    if contains(s, "negative")
        push(high, t)
    else
        push(normal, t)
    end
end

print("HIGH PRIORITY: " + len(high))
for t in high
    print("  " + t)
end
print("NORMAL: " + len(normal))
```

12.4 Conversational memory

A chat loop that remembers the user's name across sessions. The memory store persists between runs; the next time you launch the script it greets you personally.

```
# remember_me.bor
mem = memory_new("chat.db")
name = memory_recall(mem, "user_name")

if name == nil
    print("I do not know you yet. What is your name?")
    name = "(type your name)"
    memory_save(mem, "user_name", name)
else
    print("Welcome back, " + name + "!")
end
```

```
count = memory_recall(mem, "visit_count")
if count == nil
    count = 0
end
count = count + 1
memory_save(mem, "visit_count", count)
print("This is visit number " + count)
```

12.5 Simple linear-regression benchmark

Measure how well a linear model fits your data by computing R^2 on a held-out validation set. This recipe shows how to combine the ML built-ins with list manipulation.

```
# train_test.bor
x_all = range(1, 101)
y_all = []
for v in x_all
    push(y_all, 2 * v + 3 + random(0, 5))
end

# Split 80/20
cutoff = 80
x_train = []
y_train = []
x_val = []
y_val = []
i = 0
for x in x_all
    if i < cutoff
        push(x_train, x)
        push(y_train, y_all[i])
    else
        push(x_val, x)
        push(y_val, y_all[i])
    end
    i = i + 1
end

model = linear_regression(x_train, y_train)
print("Training R² = " + round(model.r2, 4))
print("slope = " + round(model.slope, 3))
print("intercept = " + round(model.intercept, 3))
```

12.6 Document classifier

Classify a document by finding the closest match among a set of labeled examples. This is a crude but surprisingly effective baseline that requires no training beyond indexing.

```
# classifier.bor
store = vector_create(128)

# Prototype documents for each class
examples = {
    sports: "football basketball tennis soccer athletics",
    tech:   "python programming server database api",
    food:   "pizza pasta salad recipe kitchen"
}

for label in keys(examples)
```

```
    vector_add(store, examples[label] + " [[" + label + "]]")
end

def classify(text)
    hit = vector_search(store, text, 1)[0]
    parts = split(hit["text"], "[[")
    label_with_suffix = parts[1]
    label = split(label_with_suffix, "]]")[0]
    return label
end

print(classify("I love cooking spaghetti with tomato sauce"))
print(classify("the server is down and the api returns 500"))
print(classify("the team scored in the last minute"))
```

12.7 Best practices

Before closing the book, a few final thoughts on using BoRLang well.

- Keep flows small and focused. If a flow has more than 15 blocks, consider decomposing it into sub-flows called via **action** handlers.
- Name variables and blocks meaningfully. **ctx["data"]** is less useful than **ctx["raw_posts"]** when debugging.
- Use the memory sandbox for conversation state, not for data that does not need to persist. Globals in the interpreter are fine for short-lived values.
- Test LLM blocks with stubs first. Build your flow with a **python** handler that returns a fixed response, then swap in the real **llm** block once the logic is correct.
- Handle HTTP errors explicitly. The **http** block returns a status code; add a condition right after to branch on failures.
- Read the trace. After every run, **flow_trace** tells you exactly which blocks ran, in what order, and how long each took. This is the single most useful debugging tool.

12.8 Closing thoughts

BoRLang started as a weekend experiment and grew into a toolkit that can genuinely help with machine learning prototyping and agent-building. It is not a replacement for Python, NumPy, or n8n — it is a smaller, sharper alternative for situations where those tools would be overkill.

The language is open to contributions. New standard-library functions, new BoRFlow block types, improvements to the parser — all are welcome. The codebase is small enough that you can read it cover to cover in an afternoon, which makes it an ideal project for learning how interpreters work.

Finally, remember the principle that guided the redesign from version 2 to version 3: **ship what works, document what does not**. This book tries to live up to that. Every example in it runs; every claim about the library's capabilities is backed by a test. If something fails to match your experience, file an issue — the honesty principle only works if we keep each other accountable.

— L.L.B && D.B, BoRLang project, 2026

11+ BoRFlow Advanced Patterns

Chapter 11 covered the fundamentals of BoRFlow. This companion chapter collects more advanced patterns that emerge once you start building agents seriously. None of what follows is strictly necessary to use BoRFlow, but every pattern here has been useful at least once in real projects, and many of them solve the kinds of problems that are not obvious until you encounter them in practice.

11A.1 Error handling and retries

Real agents fail. Network requests time out, LLM servers return garbage, APIs return the wrong status code. A robust flow must anticipate this. BoRFlow's execution model surfaces failures by raising a runtime error, which aborts the whole flow unless caught. There are three main ways to handle failures gracefully: condition-based branching, retry loops, and Python-level **try/except** at the call site.

The simplest approach is to use an **http** block, check its status code with a condition, and route accordingly:

```
f = flow_new("robust_fetch")

flow_add(f, "get", "http", {
    url: "https://api.example.com/data",
    method: "GET"
})

flow_add(f, "ok?", "condition", {
    left: "{{get.status}}", op: "==", right: 200
})

flow_add(f, "process", "transform", {
    field: "get.body.items"
})

flow_add(f, "handle_error", "print", {
    message: "Fetch failed with status {{get.status}}"
})

flow_connect(f, "get", "ok?")
flow_branch(f, "ok?", "process", "handle_error")
```

For retries on transient failures, wrap the call in a BoRLang loop at the call site rather than trying to encode retry logic inside the flow itself. This keeps the flow simple and moves the retry policy to the orchestration layer where it belongs:

```
def fetch_with_retry(url, max_attempts)
    attempt = 1
    while attempt <= max_attempts
        f = flow_new("attempt")
        flow_add(f, "get", "http", {url: url, timeout: 10})
        result = flow_run(f)
        if result["get"]["status"] == 200
            return result["get"]["body"]
```

```
        end
        print("Attempt " + attempt + " failed, retrying...")
        sleep(2 * attempt)   # exponential backoff
        attempt = attempt + 1
    end
    print("Giving up after " + max_attempts + " attempts")
    return nil
end
```

11A.2 Composing sub-flows

A single flow with twenty blocks is usually harder to read and debug than five flows with four blocks each, composed together. BoRFlow encourages this decomposition through the **action** block type, which can dispatch to any Python callable — including one that runs another flow.

```
# Define a reusable summarization sub-flow once
def make_summarizer()
    f = flow_new("summarize")
    flow_add(f, "ask", "llm", {
        model: "qwen2.5:14b",
        prompt: "Summarize in one sentence: {{text}}"
    })
    return f
end

summarizer = make_summarizer()

# A handler that invokes the sub-flow
def run_summarizer(config, ctx)
    input_text = ctx[config["from"]]
    result = flow_run(summarizer, {text: input_text})
    return result["ask"]["response"]
end

# Use it as a block in a parent flow
parent = flow_new("parent")
flow_register(parent, "summ", run_summarizer)

flow_add(parent, "step1", "action", {
    name: "summ", from: "raw_input"
})
```

This pattern scales to arbitrarily deep composition. Keep sub-flows focused on a single concern — one does retrieval, one does summarization, one does formatting — and the parent becomes a clear high-level description of the whole agent.

11A.3 Streaming versus batch

The **llm** block currently waits for the complete response before returning. For long generations, this adds noticeable latency to the user. A workaround is to use a Python action handler that calls Ollama with **stream: true** and prints tokens as they arrive:

```
def stream_llm(config, ctx)
    import json
    import urllib.request
    payload = {
        "model": config["model"],
        "prompt": config["prompt"],
```

```
        "stream": True
    }
    req = urllib.request.Request(
        "http://localhost:11434/api/generate",
        data=json.dumps(payload).encode(),
        headers={"Content-Type": "application/json"},
        method="POST"
    )
    full = ""
    with urllib.request.urlopen(req) as resp:
        for line in resp:
            chunk = json.loads(line)
            token = chunk.get("response", "")
            print(token, end="", flush=True)
            full = full + token
    return full
```

Register this as an action handler and your agent can stream tokens to the terminal in real time while still capturing the full response in the context for downstream blocks. The native **llm** block will gain streaming support in version 3.3.

11A.4 Rate limiting

Commercial APIs impose rate limits that will trip you up if you fire requests as fast as the LLM can consume them. A simple token-bucket limiter, implemented once as a closure, can be reused across any flow:

```
def make_limiter(requests_per_minute)
    interval = 60.0 / requests_per_minute
    last_call = 0
    def wait()
        now = time()
        elapsed = now - last_call
        if elapsed < interval
            sleep(interval - elapsed)
        end
        last_call = time()
    end
    return wait
end

# Create one limiter for the whole agent
limiter = make_limiter(10)   # max 10 req/min

# Use it before each LLM call
for item in items
    limiter()
    result = flow_run(llm_flow, {text: item})
end
```

11A.5 Persisting and replaying flows

A flow can be serialized to JSON with **flow_to_json** and saved to disk. This is useful for three things: sharing a flow definition with a collaborator, rendering the flow in the visual designer, and version-controlling your agent's behavior alongside its configuration data.

```
# Save
json = flow_to_json(my_flow)
```

```
write("agent.bflow.json", json)

# ... later, or in another program ...

# (Load logic would use a companion flow_from_json function,
# which is on the roadmap for v3.2)
```

NOTE. Version 3.1 only supports one direction of serialization (flow to JSON). The reverse direction — loading a flow definition from a JSON file — is planned for v3.2 once the schema has been stable for a few releases.

11A.6 Observability with trace analysis

The **flow_trace** function returns a structured log that you can analyze programmatically. This lets you build your own performance dashboards, error reports, and cost estimates without external monitoring tools.

```
def analyze_run(f)
    trace = flow_trace(f)
    total_ms = 0
    by_type = {}
    errors = 0
    for step in trace
        total_ms = total_ms + step["duration_ms"]
        t = step["type"]
        if by_type[t] == nil
            by_type[t] = 0
        end
        by_type[t] = by_type[t] + step["duration_ms"]
        if step["status"] == "error"
            errors = errors + 1
        end
    end
    print("Total time:  " + total_ms + " ms")
    print("Errors:      " + errors)
    print("Breakdown:")
    for t in keys(by_type)
        print("  " + t + ": " + by_type[t] + " ms")
    end
end

flow_run(my_flow)
analyze_run(my_flow)
```

Running this after each invocation gives you a clear picture of where time goes. Typically you will see LLM calls dominate — often 90 percent or more of the total runtime. This insight tells you exactly where optimization effort will pay off: caching LLM responses, switching to a smaller model, or running multiple LLM calls in parallel.

11A.7 Testing flows

Flows should be tested like any other code. The trick is that real LLM and HTTP calls are slow, expensive, and nondeterministic, which makes them unsuitable for an automated test suite. The solution is to use **python** action handlers as stubs: during testing, swap in a handler that returns a fixed response instead of making the real call.

```
def make_flow_with_stub_llm(stub_response)
```

```
    f = flow_new("testable")
    def stub(ctx)
        return {response: stub_response}
    end
    flow_register(f, "mock_llm", stub)
    flow_add(f, "ask", "action", {name: "mock_llm"})
    flow_add(f, "check", "condition", {
        left: "{{ask.response}}", op: "contains", right: "yes"
    })
    flow_add(f, "pos", "print", {message: "positive"})
    flow_add(f, "neg", "print", {message: "negative"})
    flow_connect(f, "ask", "check")
    flow_branch(f, "check", "pos", "neg")
    return f
end

# Test happy path
f1 = make_flow_with_stub_llm("yes, definitely")
ctx = flow_run(f1)
# assert somehow that 'pos' was printed

# Test negative path
f2 = make_flow_with_stub_llm("no, not at all")
ctx = flow_run(f2)
```

11A.8 Caching expensive blocks

LLM calls are slow and often expensive. If a flow calls the same model with the same prompt twice, the second call should return the cached result immediately. BoRFlow does not provide caching as a built-in — different use cases have different cache-invalidation needs — but you can build a general-purpose cache in about 20 lines using the memory module.

```
cache = memory_new("llm_cache.db")

def cached_llm_handler(config, ctx)
    # Build a cache key from the prompt content
    prompt = config["prompt"]
    # Render templates against ctx first
    for k in keys(ctx)
        prompt = replace(prompt, "{{" + k + "}}", str(ctx[k]))
    end
    key = sha256(config["model"] + "|" + prompt)
    cached = memory_recall(cache, key)
    if cached != nil
        print("[cache hit] " + sub(key, 0, 8))
        return cached
    end
    # Cache miss — call the real LLM
    import_time = time()
    # ... dispatch to a real llm call here via an action ...
    result = {response: "(real LLM output)"}
    memory_save(cache, key, result)
    return result
end
```

Cache keys are content-addressed hashes of the model name and rendered prompt, so identical inputs always hit the same cache entry. This works well for deterministic use cases (factual queries, summarization of stable documents) and poorly for creative tasks where you want variety. For the latter, include a random salt or timestamp in the cache key so every call is a miss.

Cache expiration is a choice. A simple policy is "always valid" — fine for immutable prompts. A more sophisticated policy includes a timestamp in the cached value and considers it stale after some duration. The memory module gives you all the tools to implement whatever policy fits your use case.

11A.9 Dynamic flow construction

Because flows are just data structures built by function calls, you can construct them dynamically based on runtime conditions. This is how you build **adaptive agents** whose structure changes depending on what the user asks for. The technique is straightforward: write a function that takes a description of what the agent should do and returns a configured flow.

```
def build_agent_for_task(task_type)
    f = flow_new("dynamic")
    if task_type == "summarize"
        flow_add(f, "do", "llm", {
            model: "qwen2.5:7b",
            prompt: "Summarize in 3 bullets: {{text}}"
        })
    elif task_type == "translate"
        flow_add(f, "do", "llm", {
            model: "qwen2.5:7b",
            prompt: "Translate to Italian: {{text}}"
        })
    elif task_type == "classify"
        flow_add(f, "do", "llm", {
            model: "qwen2.5:7b",
            prompt: "Classify as positive/neutral/negative: {{text}}"
        })
    else
        flow_add(f, "do", "print", {
            message: "Unknown task: " + task_type
        })
    end
    return f
end

# Use it
agent = build_agent_for_task("summarize")
result = flow_run(agent, {text: some_document})
```

Dynamic construction combines naturally with the action block type. You can have a top-level flow that inspects the user's request, and dispatches to dynamically-constructed sub-flows depending on the intent. This pattern is how production multi-skill agents are usually built.

11A.10 Debugging flows

When something goes wrong in a flow, the trace is your best friend. But there are a few other techniques worth knowing about.

Print-inject. Add extra **print** blocks between your real blocks to dump intermediate state. Remove them when the flow is working. This is the BoRFlow equivalent of **printf** debugging.

Run with a single block at a time. Comment out all edges except one, run the flow, inspect the resulting context, uncomment the next edge, run again. This makes it easy to see exactly which block introduces a problem.

Check the context after every block. After **flow_run** returns, the context dictionary contains every block's output keyed by block ID. If **ctx["foo"]** is unexpected, check the block's config and its upstream dependencies. Every block's logic is pure (same inputs → same output) with the exception of **http** and **llm**, so debugging is usually a matter of tracking down where the first wrong value appears.

Validate LLM outputs before using them. If a block expects the LLM to return JSON, wrap the parsing in a condition check and handle malformed responses. LLMs occasionally add markdown fences, preamble, or extra commentary around their output, and flows that assume clean JSON will fail mysteriously when this happens.

Save and replay. Record the full context and trace of a failing run to disk. You can then reconstruct the state that led to the failure and iterate on a fix without re-running the upstream network calls.

11A.11 Measuring agent quality

The hardest part of building real agents is not making them work at all but making them work consistently. A prompt that produces a correct answer nine times out of ten needs to be measured against one that produces a correct answer ten times out of ten. This is the domain of **evaluation**, and BoRLang gives you the primitives to do it.

The simplest evaluation setup is a list of test cases, each with an input and an expected output. Run the agent on every input and compare. Track the fraction that match. Here is a minimal evaluator:

```
cases = [
    {input: "What is 2+2?", expected: "4"},
    {input: "Capital of Italy?", expected: "Rome"},
    {input: "Largest ocean?", expected: "Pacific"}
]

correct = 0
for c in cases
    result = flow_run(agent, {question: c["input"]})
    answer = lower(trim(result["reply"]["response"]))
    if contains(answer, lower(c["expected"]))
        correct = correct + 1
        print("PASS: " + c["input"])
    else
        print("FAIL: " + c["input"])
        print("   expected: " + c["expected"])
        print("   got:      " + answer)
    end
end
score = correct / len(cases)
print("Score: " + round(score * 100, 1) + "%")
```

Exact-match comparison is crude — many right answers will be worded differently than your expected string. More robust evaluators use a second LLM to judge whether the response is correct (the "LLM-as-judge" pattern), or compare embeddings to check semantic similarity. Both are straightforward to implement in BoRFlow: the judge is just another flow, the similarity check is a **similarity** call on the embeddings.

Evaluation is what separates toy agents from production ones. The agents you see deployed successfully in the wild are the ones their developers measured ruthlessly and iterated on based on failures. BoRFlow's explicit structure makes this iteration easier than it would be with a more opaque

framework, because you can swap individual blocks and immediately measure the effect.

13 Tutorial: Five Projects

The previous chapters introduced BoRLang and BoRFlow feature by feature. This chapter puts those features together into five complete, working projects that progress from very simple to moderately sophisticated. Each project is self-contained: you can type it into a file and run it without reading the others first. The projects are, however, designed to build on each other conceptually, so you will get the most out of the chapter by working through them in order.

The five projects are: a text-analysis command-line tool, a personal note-taking agent with persistent memory, a semantic search engine over a folder of text files, an automated research assistant that combines web fetching with LLM summarization, and a multi-agent task delegator that coordinates three specialized sub-agents to solve a single problem. Each subsequent project introduces one or two new techniques while reusing everything that came before.

A note on the code listings. Many of the examples in this chapter are long enough that they span multiple pages when typeset. Each listing begins with a comment giving the filename, and the listings are unbroken by prose so you can copy them in one go. After each listing, a commentary section explains what the code is doing and why it is structured that way.

13.1 Project 1: Text analyzer

Our first project is a self-contained command-line tool that reads a text file and reports word count, character count, unique word count, average word length, the ten most frequent words, and the estimated reading time. It uses only features from chapters 2 through 6, so you can treat it as a capstone exercise for the language-basics half of the book. The point is to show how much you can do without any of the specialized modules.

Start by creating a file called **analyzer.bor**. The high-level structure of the program is: read the input file, tokenize it into words, collect statistics, and print a formatted report. Each of these steps is a small helper function, and the main body at the bottom chains them together.

```
# analyzer.bor — text analysis tool
# Usage: python borlang.py analyzer.bor <filename>

# --- Helpers ---

def tokenize(text)
    # Lowercase, strip punctuation, split on whitespace
    clean = lower(text)
    clean = gsub(clean, "[^a-z0-9 ]", " ")
    words = []
    for w in split(clean, " ")
        if len(w) > 0
            push(words, w)
        end
    end
    return words
end

def count_frequencies(words)
```

```
    freq = {}
    for w in words
        existing = freq[w]
        if existing == nil
            freq[w] = 1
        else
            freq[w] = existing + 1
        end
    end
    return freq
end

def top_n(freq, n)
    # Convert dict to list of [word, count] pairs and sort
    pairs = []
    for w in keys(freq)
        push(pairs, [w, freq[w]])
    end
    # Bubble sort by count descending
    i = 0
    while i < len(pairs)
        j = i + 1
        while j < len(pairs)
            if pairs[j][1] > pairs[i][1]
                temp = pairs[i]
                pairs[i] = pairs[j]
                pairs[j] = temp
            end
            j = j + 1
        end
        i = i + 1
    end
    # Take first n
    result = []
    i = 0
    while i < n and i < len(pairs)
        push(result, pairs[i])
        i = i + 1
    end
    return result
end

def avg_word_length(words)
    if len(words) == 0
        return 0
    end
    total = 0
    for w in words
        total = total + len(w)
    end
    return total / len(words)
end
```

The four helpers each do one thing. **tokenize** normalizes the text (lowercase and strip anything that is not a letter or digit), then splits on spaces and filters out empty fragments. **count_frequencies** builds a dictionary mapping each unique word to its occurrence count; the **nil** check is how we distinguish a first sighting from a repeat. **top_n** extracts the N most common words by turning the dictionary into a list of pairs and sorting it with a handwritten bubble sort — BoRLang's **sort** built-in only handles flat lists of comparable values, so for custom orderings we roll our own. **avg_word_length** is self-explanatory.

Now the main body that wires everything together:

```
# --- Main ---

filename = "sample.txt"      # change to your file
text = read(filename)
if text == nil
    print("Error: could not read " + filename)
    return
end

words = tokenize(text)
freq = count_frequencies(words)
top10 = top_n(freq, 10)
avg = avg_word_length(words)

# Reading time assumes 200 words per minute
read_minutes = round(len(words) / 200, 1)

print("=== Text Analysis: " + filename + " ===")
print("Total characters:    " + len(text))
print("Total words:         " + len(words))
print("Unique words:        " + len(keys(freq)))
print("Average word length: " + round(avg, 2))
print("Reading time:        ~" + read_minutes + " min")
print("")
print("Top 10 words:")
rank = 1
for pair in top10
    word = pair[0]
    count = pair[1]
    print("  " + rank + ". " + word + " (" + count + ")")
    rank = rank + 1
end
```

Save any paragraph of English text as **sample.txt** in the same folder and run the program. The output is a report that looks roughly like this:

```
=== Text Analysis: sample.txt ===
Total characters:    2847
Total words:         487
Unique words:        253
Average word length: 4.72
Reading time:        ~2.4 min

Top 10 words:
  1. the (32)
  2. of (18)
  3. a (15)
  4. and (12)
  5. to (11)
  6. in (9)
  7. is (8)
  8. with (6)
  9. that (6)
  10. for (5)
```

NOTE. This first project deliberately avoids any external libraries or specialized modules. Everything you see is core BoRLang from chapters 2 through 6. That the program is still reasonably concise is evidence the core language holds its own weight.

Possible extensions for the reader: compute the **type/token ratio** (unique words divided by total words, a rough proxy for vocabulary richness), detect and exclude stopwords so the top-ten list shows meaningful content words instead of articles and prepositions, or add a command-line argument parser so the filename can come from **sys.argv** rather than being hard-coded.

13.2 Project 2: Personal notes agent

Our second project builds a persistent note-taking tool. The user runs the script with a command (**add**, **list**, **find**, or **delete**) followed by some input text. Notes are stored in a SQLite-backed memory store from chapter 9, so they survive between runs. Each note has an auto-incrementing numeric ID, a timestamp, and a free-form body.

This project introduces the pattern of a small **command dispatcher** — a conditional block that routes to different functions based on an input keyword. This pattern appears throughout agentic programs, so it is worth learning here.

```
# notes.bor — persistent note-taking agent
# Usage (edit the bottom of the script to change command):
#   add:    put a new note
#   list:   show all notes
#   find:   filter notes by substring
#   delete: remove a note by ID

mem = memory_new("notes.db")

def next_id()
    last = memory_recall(mem, "__next_id__")
    if last == nil
        last = 1
    else
        last = last + 1
    end
    memory_save(mem, "__next_id__", last)
    return last
end

def note_key(id)
    return "note_" + id
end

def add_note(text)
    id = next_id()
    memory_save(mem, note_key(id), {
        id: id,
        text: text,
        created: datetime()
    })
    print("Added note #" + id)
    return id
end

def load_all_notes()
    notes = []
    # We don't have a key-iteration primitive in memory, so we
    # scan IDs from 1 up to the current max.
    max_id = memory_recall(mem, "__next_id__")
    if max_id == nil
        return notes
    end
    i = 1
    while i <= max_id
        n = memory_recall(mem, note_key(i))
        if n != nil
            push(notes, n)
        end
```

```
        i = i + 1
    end
    return notes
end

def list_notes()
    notes = load_all_notes()
    if len(notes) == 0
        print("(no notes yet)")
        return
    end
    print("=== " + len(notes) + " note(s) ===")
    for n in notes
        print("#" + n["id"] + "  [" + n["created"] + "]")
        print("   " + n["text"])
    end
end

def find_notes(query)
    notes = load_all_notes()
    matches = []
    for n in notes
        if contains(lower(n["text"]), lower(query))
            push(matches, n)
        end
    end
    print("Found " + len(matches) + " match(es) for '" + query + "':")
    for n in matches
        print("#" + n["id"] + "  " + n["text"])
    end
end

def delete_note(id)
    if memory_forget(mem, note_key(id))
        print("Deleted note #" + id)
    else
        print("No note with id " + id)
    end
end

# --- Entry point: edit these two lines ---
command = "add"
argument = "Remember to buy milk and bread tomorrow"

if command == "add"
    add_note(argument)
elif command == "list"
    list_notes()
elif command == "find"
    find_notes(argument)
elif command == "delete"
    delete_note(int(argument))
else
    print("Unknown command: " + command)
end
```

Run the script once with **command = "add"**; then change the command to **"list"** and run again. The note persists because it was saved to **notes.db**. A small detail worth noticing: the function **next_id** uses the memory store itself to remember what the next ID should be, under the special key **__next_id__**. This is a common pattern — metadata about a store lives in reserved keys alongside the actual entries.

The **load_all_notes** function has a limitation that is worth being honest about: it walks from ID 1 upward, which is fine for a few hundred notes but becomes slow at scale. A real implementation would maintain a separate index list, or would use SQLite directly via a Python action handler. Version 3.2 of BoRLang plans to add a **memory_keys** primitive that would make this efficient.

Extensions for the reader: add tags to each note and filter by tag, add an **edit** command that replaces the text of an existing note, persist a simple full-text search index so **find** does not need to scan every note, or extend the note structure with optional due dates and add a **today** command that lists notes due today.

13.3 Project 3: Folder semantic search

Our third project builds a miniature search engine. Point it at a folder of text files and it will index every file into a vector store, then let you run natural-language queries that return the three most relevant passages along with their source filenames. This is the same core idea behind modern document search tools like Obsidian's semantic search or Notion's AI features, just in 60 lines of BoRLang.

Two new techniques appear here. First, **filesystem walking** using the **ls** built-in to enumerate files in a directory. Second, **chunked indexing**: rather than embedding each entire file as a single vector, we split files into paragraphs and embed each paragraph separately, so the search can point to exactly which part of which file matches.

```
# search.bor — semantic search over a folder of text files

folder = "./docs"

store = vector_create(256)
sources = []    # parallel list: sources[i] is the filename of vector i

def index_file(path)
    content = read(path)
    if content == nil
        return 0
    end
    paragraphs = split(content, "\n\n")
    indexed = 0
    for p in paragraphs
        cleaned = trim(p)
        if len(cleaned) >= 50
            vector_add(store, cleaned)
            push(sources, path)
            indexed = indexed + 1
        end
    end
    return indexed
end

def index_folder(path)
    files = ls(path)
    if files == nil
        print("Folder not found: " + path)
        return
    end
    print("Scanning " + path + "...")
    total = 0
    for f in files
        if endswith(f, ".txt") or endswith(f, ".md")
            full = path + "/" + f
            n = index_file(full)
            print("  " + f + ": " + n + " passages")
            total = total + n
        end
    end
    print("Total passages indexed: " + total)
end

def search(query, k)
    print("")
    print("=== Query: '" + query + "' ===")
```

```
    hits = vector_search(store, query, k)
    rank = 1
    for h in hits
        source = sources[h["id"]]
        print(rank + ". [" + round(h["score"], 3) +
              "] from " + source)
        # Show first 120 chars of the passage
        preview = h["text"]
        if len(preview) > 120
            preview = sub(preview, 0, 120) + "..."
        end
        print("   " + preview)
        print("")
        rank = rank + 1
    end
end

# --- Run ---
index_folder(folder)

# Try a few queries
search("machine learning algorithms", 3)
search("how to write fast code", 3)
search("vector databases", 3)
```

The **sources** list is a parallel array: its i-th entry tells you which file the i-th indexed vector came from. This is a classic trick for keeping extra metadata alongside a data structure that does not support metadata natively. BoRLang's vector store indexes vectors by integer ID in insertion order, so **sources[hit.id]** is guaranteed to point to the right file.

The **search** function does something subtle when showing results: it truncates long passages to 120 characters so the output stays scannable. This is a user-experience detail that matters in real tools — showing full passages quickly fills the screen and hides the relative ranking.

> **NOTE.** If you install **sentence-transformers** the search quality improves dramatically. With the default hash embedder, your queries need to share vocabulary with the indexed text. With a real semantic embedder, "how to write fast code" might correctly match a passage titled "performance optimization techniques" even though they share no words.

Extensions for the reader: add a cache so repeated runs do not re-index files that have not changed (use the file modification time in the memory store as the cache key), support PDF files via a Python action handler that extracts text, or add a query-expansion step that asks a local LLM to generate synonyms before searching.

13.4 Project 4: Automated research assistant

Our fourth project is the first that truly combines BoRLang with BoRFlow. Given a list of URLs and a research question, the assistant fetches each page, extracts the main text, summarizes it with a local LLM, and synthesizes a combined answer. It is the simplest form of what commercial products market as "AI research agents" — and seeing it implemented in under a hundred lines demystifies what those products are doing under the hood.

This project assumes you have **Ollama** running locally with a chat-capable model pulled — any of **llama3.2**, **qwen2.5**, or **mistral** will work. If you do not have Ollama set up, skip the last phase and look at the summaries of each individual page; the structure of the agent is the educational part.

```
# research.bor — multi-source research assistant

question = "What are the main principles of functional programming?"

urls = [
    "https://en.wikipedia.org/wiki/Functional_programming",
    "https://example.com/fp-intro",
    "https://example.com/fp-patterns"
]

model = "qwen2.5:14b"

# --- Step 1: build a per-source flow ---

source_flow = flow_new("fetch_and_summarize")

flow_add(source_flow, "get", "http", {
    url: "{{url}}",
    method: "GET",
    timeout: 30
})

flow_add(source_flow, "summarize", "llm", {
    model: model,
    prompt: "Given the user's research question:\n" +
            "\"{{question}}\"\n\n" +
            "Summarize the following web page in 3-4 bullet " +
            "points, focusing only on content relevant to the " +
            "question. Ignore navigation, ads, and unrelated " +
            "material.\n\n{{get.body}}",
    timeout: 120
})

flow_connect(source_flow, "get", "summarize")

# --- Step 2: run the flow for each URL ---

summaries = []

for url in urls
    print("Fetching " + url)
    result = flow_run(source_flow, {
        url: url,
        question: question
    })
    # Record the summary with its source
    summary = result["summarize"]["response"]
```

```
    push(summaries, {
        url: url,
        summary: summary
    })
    print("  summary length: " + len(summary) + " chars")
end

# --- Step 3: synthesize a combined answer ---

synthesis_flow = flow_new("synthesize")

# Build the combined context for the final LLM call
context = ""
for s in summaries
    context = context + "Source: " + s["url"] + "\n"
    context = context + s["summary"] + "\n\n"
end

flow_add(synthesis_flow, "combine", "llm", {
    model: model,
    prompt: "You have gathered the following notes on \"" +
            question + "\":\n\n" + context +
            "\nWrite a single, coherent answer to the question. " +
            "Cite each claim with the source URL in parentheses. " +
            "If the sources disagree, note the disagreement.",
    timeout: 180
})

final = flow_run(synthesis_flow)

print("")
print("=== FINAL ANSWER ===")
print(final["combine"]["response"])
```

The architecture is three distinct phases — fetch, summarize, synthesize — each implemented as a BoRFlow flow or a loop over one. Breaking it up this way has two benefits. First, if something fails (for example, one of the URLs returns a 404), only that source is affected and the rest of the pipeline continues. Second, each phase can be tested independently: you can run the fetch phase, save the summaries to disk, and develop the synthesis phase using the saved data, avoiding repeated network calls and LLM invocations during iteration.

Notice how the same **source_flow** is reused across all URLs by calling **flow_run** three times with different initial contexts. This is the power of flows as reusable units: the logic is defined once, then parameterized per invocation. It is the same pattern as a function, but at the level of multi-step agentic pipelines.

> **WARNING.** Running this project hits real LLM inference and real HTTP endpoints. Be considerate: do not fetch hundreds of URLs in a tight loop, respect robots.txt on sites you do not own, and remember that LLM calls consume time and GPU memory. For development, test with two or three URLs before scaling up.

A subtle improvement you can add with about ten extra lines: after fetching each page, use **compress_zlib** to save the raw HTML to disk, keyed by a hash of the URL. The next time you research the same question you can load the cached pages instead of refetching. This makes iteration much faster during development of the summarization prompt.

Extensions: add a condition block that checks the HTTP status code and skips failed fetches, implement prompt caching so repeated summarization of the same page returns instantly, or add a final **fact-check** phase that asks the LLM to identify any claims in the synthesis not supported by the

summaries.

13.5 Project 5: Multi-agent delegator

Our final project is the most ambitious. It implements a pattern called **multi-agent coordination**: a single user request is decomposed into sub-tasks that are handled by specialized sub-agents, then their outputs are combined into a unified response. The specific scenario we will build is a travel planner. The user asks for a trip recommendation, and three sub-agents — a destination researcher, a budget estimator, and a logistics planner — each contribute to the final itinerary.

Multi-agent systems are currently a hot research topic because they let you decompose hard problems into pieces each LLM call can handle well. Instead of asking one mega-prompt to do everything, you split the work along natural seams and give each sub-agent a focused prompt. The trade-off is orchestration complexity: you have to define how the sub-agents communicate and how their outputs merge. BoRFlow's explicit flow structure makes this orchestration visible and version-controllable.

For clarity, we will define three flows — one per sub-agent — and a main coordinator that runs them and combines their outputs. Each sub-agent uses the same underlying LLM but with a different system prompt that casts it into a different role.

```
# delegator.bor — three specialized sub-agents working together

model = "qwen2.5:14b"
user_request = "Plan a 5-day trip to Japan for two people in April"

# --- Agent 1: Destination researcher ---

def make_researcher()
    f = flow_new("researcher")
    flow_add(f, "think", "llm", {
        model: model,
        system: "You are a travel destination expert. Given a " +
                "trip request, suggest 3-5 cities or regions to " +
                "visit, with one sentence of reasoning per choice. " +
                "Be concrete and opinionated.",
        prompt: "{{request}}"
    })
    return f
end

# --- Agent 2: Budget estimator ---

def make_budgeter()
    f = flow_new("budgeter")
    flow_add(f, "think", "llm", {
        model: model,
        system: "You are a travel budget expert. Given a trip " +
                "request and a list of destinations, estimate " +
                "total cost for flights, accommodation, food, " +
                "and transit in USD. Give a range (min/max) and " +
                "note what drives the variation.",
        prompt: "Trip request: {{request}}\n\n" +
                "Proposed destinations:\n{{destinations}}"
    })
    return f
end

# --- Agent 3: Logistics planner ---
```

```
def make_planner()
    f = flow_new("planner")
    flow_add(f, "think", "llm", {
        model: model,
        system: "You are a travel logistics expert. Given a trip " +
                "and a list of destinations, produce a day-by-day " +
                "itinerary. Specify morning, afternoon, and " +
                "evening activities for each day. Include " +
                "transit between cities. Be realistic about " +
                "time and energy.",
        prompt: "Trip: {{request}}\n\n" +
                "Destinations:\n{{destinations}}\n\n" +
                "Budget context:\n{{budget}}"
    })
    return f
end

# --- Coordinator ---

print("Dispatching to researcher...")
researcher = make_researcher()
r_result = flow_run(researcher, {request: user_request})
destinations = r_result["think"]["response"]
print("[researcher] -> " + len(destinations) + " chars")

print("Dispatching to budgeter...")
budgeter = make_budgeter()
b_result = flow_run(budgeter, {
    request: user_request,
    destinations: destinations
})
budget = b_result["think"]["response"]
print("[budgeter] -> " + len(budget) + " chars")

print("Dispatching to planner...")
planner = make_planner()
p_result = flow_run(planner, {
    request: user_request,
    destinations: destinations,
    budget: budget
})
itinerary = p_result["think"]["response"]
print("[planner] -> " + len(itinerary) + " chars")

# --- Combine into final report ---

print("")
print("============================================")
print("         TRAVEL PLAN")
print("============================================")
print("")
print("REQUEST: " + user_request)
print("")
print("--- Destinations ---")
print(destinations)
print("")
print("--- Budget ---")
print(budget)
print("")
print("--- Day-by-day ---")
print(itinerary)
```

```
# Save the complete plan for later reference
write("travel_plan.txt",
    "REQUEST: " + user_request + "\n\n" +
    "DESTINATIONS\n" + destinations + "\n\n" +
    "BUDGET\n" + budget + "\n\n" +
    "ITINERARY\n" + itinerary)

print("")
print("Plan saved to travel_plan.txt")
```

Three things in this project are worth studying closely. First, each sub-agent is a **factory function** that returns a fresh flow. This lets you create multiple independent instances of the same agent type, for example if you wanted to ask two researchers with different temperatures and compare their answers. Second, the sub-agents form a **pipeline**: the researcher's output becomes part of the budgeter's input, and both of their outputs feed into the planner. Third, the coordinator is just a plain BoRLang script — no special framework is needed to wire the agents together, because **flow_run** returns a regular context dictionary.

The pattern generalizes to many other tasks. Replace the three travel sub-agents with a spec writer, a coder, and a tester and you have a prototype of a software-development agent. Replace them with an ingredient proposer, a recipe writer, and a nutritionist and you have a meal-planning agent. The orchestration code barely changes; only the sub-agents' system prompts differ.

> **NOTE.** For ambitious readers, consider a more advanced pattern: after the researcher proposes destinations, run the budgeter and planner **in parallel** rather than sequentially. BoRFlow 3.1 does not yet support parallel execution natively, but you can achieve the same effect by running each sub-flow in a separate Python thread from an **action** handler. This is an advanced technique and is on the roadmap for version 3.3.

13.6 What to build next

If you have worked through all five projects, you now know enough to tackle real problems with BoRLang. The next steps are up to you, but here are five concrete ideas that would each make a good weekend project.

First, a **personal journal analyzer** that reads your journal entries from a folder, clusters them by topic with k-means on the vector embeddings, and produces a month-end summary for each cluster. This exercises chapter 7 (ML), chapter 8 (vectors), and chapter 9 (memory) together.

Second, a **code review bot** that reads a pull-request diff, splits it into per-file hunks, asks the LLM to flag issues in each hunk, and combines the comments into a review. This is the research-assistant pattern from project 4, but applied to code instead of web pages.

Third, a **data-entry extractor** that takes a folder of unstructured documents (invoices, receipts, meeting notes), uses an LLM to extract structured fields from each, and writes the results to a spreadsheet. The extraction prompt is the interesting part; the rest is file walking and CSV formatting.

Fourth, a **recommendation agent** that takes your ratings of things you have tried (movies, restaurants, books), computes embeddings of the items you liked, and suggests similar things from a catalog. This is a real-world application of vector search to a problem most people actually have.

Fifth, a **daily standup bot** that reads your git commit log and your calendar, uses an LLM to summarize what you did yesterday and what you plan to do today, and posts the result to a persistent note. This combines nearly every feature in the book and makes itself useful every morning.

Whatever you build, share it. One of the hopes for BoRLang is that a small community forms around it to exchange recipes, flow types, and patterns. If you write something interesting, open a pull request to add it to the examples folder of the official repository.

14 Two More Projects

The five projects in chapter 13 covered the breadth of what BoRLang can do. This chapter goes deeper with two more projects that each solve a real and nontrivial problem end-to-end. The first is a log-file analyzer that uses ML to detect anomalies. The second is a chatbot with long-term memory that remembers facts about the user across conversations. Both are the kind of thing that people regularly pay money for; both fit in under 150 lines of BoRLang.

14.1 Project 6: Log anomaly detector

Every production server generates log files. Most of their content is routine — successful requests, heartbeats, normal errors — but occasionally something unusual appears that deserves human attention. Log monitoring tools address this by clustering similar log lines and flagging outliers. We are going to build a miniature version of this in BoRLang.

The high-level approach is: read every line of the log, convert each into a vector using the hash embedder, cluster the vectors with k-means, compute each line's distance from its cluster center, and flag the lines with the largest distances as anomalies. This is an unsupervised method — no one has to label the training data — which makes it usable for logs whose contents vary from deployment to deployment.

```
# anomaly.bor — log anomaly detector

logfile = "app.log"
k = 5              # number of normal clusters
top_n = 10         # how many anomalies to flag

# --- Step 1: load and tokenize lines ---

content = read(logfile)
if content == nil
    print("Cannot open " + logfile)
    return
end

lines = split(content, "\n")
# Filter empty and very short lines
good = []
for line in lines
    if len(trim(line)) > 20
        push(good, line)
    end
end
print("Loaded " + len(good) + " log lines")

# --- Step 2: embed each line ---

def embed(text, dim)
    # Simple hash embedding: bucket each word into a fixed vector
    vec = []
    i = 0
    while i < dim
```

```
        push(vec, 0)
        i = i + 1
    end
    words = split(lower(text), " ")
    for w in words
        if len(w) > 0
            # Use sha256 hash truncated to an int
            h = sha256(w)
            bucket = int("0x" + sub(h, 0, 6)) % dim
            vec[bucket] = vec[bucket] + 1
        end
    end
    return vec
end

vectors = []
for line in good
    push(vectors, embed(line, 64))
end
print("Computed " + len(vectors) + " embeddings")

# --- Step 3: cluster with k-means ---

result = kmeans(vectors, k)
labels = result["labels"]
centers = result["centers"]
print("Clustering done; inertia = " + round(result["inertia"], 2))

# --- Step 4: compute distance from each line to its center ---

def euclidean(a, b)
    total = 0
    i = 0
    while i < len(a)
        d = a[i] - b[i]
        total = total + d * d
        i = i + 1
    end
    return sqrt(total)
end

distances = []
i = 0
while i < len(vectors)
    cluster_id = labels[i]
    d = euclidean(vectors[i], centers[cluster_id])
    push(distances, {line: good[i], distance: d, cluster: cluster_id})
    i = i + 1
end

# --- Step 5: sort and report top-N outliers ---

# Bubble sort by distance descending (small list, this is fine)
i = 0
while i < len(distances)
    j = i + 1
    while j < len(distances)
        if distances[j]["distance"] > distances[i]["distance"]
            temp = distances[i]
            distances[i] = distances[j]
            distances[j] = temp
        end
```

```
        j = j + 1
    end
    i = i + 1
end

print("")
print("=== Top " + top_n + " anomalies ===")
i = 0
while i < top_n and i < len(distances)
    d = distances[i]
    print("[d=" + round(d["distance"], 2) +
          " cluster=" + d["cluster"] + "]")
    print("   " + d["line"])
    i = i + 1
end
```

Save any log file as **app.log** — web server access logs, application debug logs, Kubernetes events, anything with line-oriented text — and run the detector. The top ten entries will be the lines that look least like anything else in the log. In practice these tend to be error bursts, unusual request patterns, or one-off events that a human should investigate.

The quality of the detection depends strongly on the embedding. The hash embedder treats every unique word equally, so a line with an unusual word scores as anomalous even if the unusual word is benign (like a random request ID). More sophisticated real-world detectors use TF-IDF weighting, contextual embeddings from a language model, or domain-specific tokenizers that normalize things like timestamps and IP addresses. These are all good directions for extending the project.

> **NOTE.** Anomaly detection is a probabilistic technique, not a binary classifier. Expect false positives. The value of the tool is that it narrows your attention from ten thousand log lines to ten, not that every one of those ten is a genuine incident. Treat the output as a triage queue rather than a list of bugs.

Extensions to try: add a rolling window so the detector only looks at the last hour of logs, combine multiple features (HTTP status code, response time, IP address hash) into a single anomaly score, or use the memory store to remember which anomalies have already been reported so you do not alert on the same line repeatedly.

14.2 Project 7: Chatbot with long-term memory

Standard chatbot frameworks give the LLM access to the last few turns of conversation, which works well until the interesting content has scrolled out of the window. A better architecture augments short-term (in-context) memory with long-term memory: a vector store of past conversations that the agent searches for relevant context before responding. This is the pattern behind systems like ChatGPT's memory feature and various agent frameworks' memory modules.

Our implementation has three components. A **session buffer** holds the current conversation in a list. A **vector memory** indexes every previous turn for later retrieval. A **reflection step** runs after each user turn, asking the LLM to extract stable facts (the user's name, job, preferences) and saving them to a separate structured store. Together these give the bot episodic memory for recent context and semantic memory for long-lived facts.

```
# chatbot.bor — chatbot with long-term memory

model = "qwen2.5:14b"
session = []                # current conversation
long_term = vector_create(256)   # vector store of past turns
facts = memory_new("user_facts.db")  # structured fact store

# --- Helpers ---

def add_to_session(role, text)
    push(session, {role: role, text: text})
end

def session_to_prompt()
    out = ""
    for turn in session
        out = out + turn["role"] + ": " + turn["text"] + "\n"
    end
    return out
end

def retrieve_relevant(query)
    hits = vector_search(long_term, query, 3)
    out = ""
    for h in hits
        if h["score"] > 0.1
            out = out + "- " + h["text"] + "\n"
        end
    end
    return out
end

def load_known_facts()
    out = ""
    for k in ["user_name", "user_job", "user_city",
              "user_interests"]
        v = memory_recall(facts, k)
        if v != nil
            out = out + k + ": " + v + "\n"
        end
    end
    return out
end

# --- Reflection: extract and save stable facts ---
```

```
reflect_flow = flow_new("reflect")
flow_add(reflect_flow, "extract", "llm", {
    model: model,
    system: "You extract factual claims the user has made " +
            "about themselves. Output a JSON object with any " +
            "of these fields that appear: user_name, user_job, " +
            "user_city, user_interests. Output only the JSON, " +
            "no preamble. Empty object {} if nothing relevant.",
    prompt: "User just said: {{user_text}}"
})

def reflect_and_save(user_text)
    result = flow_run(reflect_flow, {user_text: user_text})
    raw = trim(result["extract"]["response"])
    # Strip fenced code block if present
    if startswith(raw, "```")
        raw = gsub(raw, "```json", "")
        raw = gsub(raw, "```", "")
        raw = trim(raw)
    end
    extracted = json_decode(raw)
    if extracted == nil
        return
    end
    for k in keys(extracted)
        memory_save(facts, k, extracted[k])
        print("[learned] " + k + " = " + extracted[k])
    end
end

# --- Main answer flow ---

answer_flow = flow_new("answer")
flow_add(answer_flow, "reply", "llm", {
    model: model,
    system: "You are a helpful assistant with persistent " +
            "memory. Known facts about the user:\n" +
            "{{known_facts}}\n\n" +
            "Relevant past conversation:\n{{past}}\n\n" +
            "Use these if helpful but do not mention them " +
            "unless asked. Be concise.",
    prompt: "{{current}}"
})

def respond(user_text)
    add_to_session("user", user_text)
    known = load_known_facts()
    past = retrieve_relevant(user_text)
    current = session_to_prompt()
    result = flow_run(answer_flow, {
        known_facts: known,
        past: past,
        current: current
    })
    reply = trim(result["reply"]["response"])
    add_to_session("assistant", reply)
    # Index this exchange for future retrieval
    vector_add(long_term, "user: " + user_text)
    vector_add(long_term, "assistant: " + reply)
    # Extract stable facts in the background
    reflect_and_save(user_text)
    return reply
```

```
end

# --- Example session ---

print("> " + respond("Hi! My name is Alessia and I'm a vet."))
print("")
print("> " + respond("I'm moving to Torino next month."))
print("")
print("> " + respond("What do you remember about me?"))
```

The architecture is worth studying. Every user turn triggers three parallel processes: the answer flow produces an immediate reply using known facts and retrieved history, the vector store indexes the current turn for future retrieval, and the reflection flow extracts stable facts and saves them to the structured store. Over many sessions, the facts store grows into a durable profile of the user while the vector store accumulates episodic memories.

A subtle design decision: the reflection flow runs **after** the reply is produced, not before. This means the first turn of a conversation does not yet know anything about the user, but subsequent turns do. If you wanted the bot to remember facts even in the very first message, you would flip the order — but then every turn would incur an extra LLM call before responding, which significantly increases latency. The tradeoff is a real product decision.

> **NOTE.** In practice, ask the LLM to extract facts with a few-shot prompt showing examples of good extractions and edge cases ("user says their name is X — save user_name; user asks what the capital of France is — save nothing"). This dramatically improves extraction quality versus a zero-shot instruction. The few-shot prompt is the place where real engineering effort goes in production chatbot systems.

Extensions that would make the bot substantially better: add a **summarization** step that compresses long-running conversations into a running summary stored alongside the facts, implement **fact conflict resolution** (when the user says their city has changed, update rather than duplicate), and add a **trust level** to each stored fact so speculative extractions are treated differently from confirmed ones.

A Quick-Reference Card

Tear this page out (or keep the PDF open in a split window) while you program. It summarizes the language essentials in a format designed for scanning rather than reading.

A.1 Literal forms

```
42              integer
3.14            float
"hello"         string (double quotes)
'world'         string (single quotes)
true false      booleans
nil             absence of value
[1, 2, 3]       list
{k: v, ...}     dictionary
```

A.2 Statements

```
x = 5                        assignment
if cond ... end              single-branch conditional
if cond ... else ... end     two-branch
if ... elif ... else ... end  multi-branch
while cond ... end           loop while true
for x in collection ... end loop over items
def name(params) ... end     function definition
return expr                  return from function
```

A.3 Operators

```
+  -  *  /  %                arithmetic
==  !=  <  >  <=  >=         comparison
and  or  not                 logical
+                            string concat (when string operand)
```

A.4 Running programs

```
python borlang.py                  # REPL
python borlang.py script.bor       # run file
python test_borlang.py             # core tests
python test_flow.py                # flow tests
```

B Glossary

action — A BoRFlow block type that dispatches to a user-registered Python handler. Useful for extending BoRFlow with custom block types.

amplitude encoding — A classical algorithm for representing a data vector as a unit-norm vector whose components can be interpreted as quantum amplitudes. Used by quantum_encode.

BoRFlow — The agentic workflow engine bundled with BoRLang. Flows are DAGs of blocks (nodes) connected by edges.

BoRLang — The scripting language itself. Python-like syntax with Lua-like scoping and no external parsing dependencies.

closure — A function that remembers the environment in which it was defined. In BoRLang, every function is a closure.

cluster — A group of similar items produced by an unsupervised learning algorithm like k-means.

context — The shared dictionary that flows through a BoRFlow execution, storing each block's result under its node ID.

embedding — A numeric vector representation of text (or any other data) chosen so that semantic similarity corresponds to vector proximity.

FAISS — Meta's similarity-search library. Optional backend for BoRLang's vector store.

flow — A directed acyclic graph of blocks and edges that defines a BoRFlow workflow.

hash embedding — A cheap, deterministic embedding that hashes each word to a bucket in a fixed-size vector. Fast but not semantic.

inertia — In k-means, the sum of squared distances of every point to its assigned cluster center. Lower is tighter.

lexical scope — A scoping rule where variable visibility is determined by the source-code structure rather than the call order at runtime.

linear regression — The simplest supervised ML algorithm: find a line (or higher-dimensional equivalent) that best fits a set of data points.

LLM — Large language model. In BoRFlow, typically called via a locally-running Ollama server.

memory sandbox — BoRLang's SQLite-backed key-value store for persisting values across runs.

node — A single block within a BoRFlow flow.

normalization — Scaling data so that different features have comparable magnitudes. BoRLang provides L2 normalization which makes every row a unit vector.

Ollama — A local LLM server (*https://ollama.ai*) used by BoRFlow's llm block type.

PCA — Principal component analysis. A dimensionality-reduction technique that finds the directions of greatest variance in the data.

RAG — Retrieval-augmented generation. A pattern where an LLM answers questions using documents retrieved by vector search.

REPL — Read-eval-print loop. The interactive BoRLang prompt.

SQLite — A lightweight embedded SQL database, used by BoRLang's memory module.

stdlib — The standard library — the set of functions that are always available in BoRLang without additional imports.

trace — The log of block executions produced by a BoRFlow run. Retrieved with flow_trace.

transform — A BoRFlow block type that extracts a field or applies a simple operation to a context value.

truthiness — The rule BoRLang uses to decide whether a value is considered true in a boolean context. False, nil, zero, empty string, and empty collections are falsy; everything else is truthy.

vector store — A data structure that indexes text as vectors and supports nearest-neighbor search. The core of semantic retrieval.

C Function Index

The following is a sorted index of every built-in function in BoRLang 3.1, with a one-line description. For full parameter documentation, see chapter 6.

`abs(n)`	absolute value
`append(path, text)`	append text to file
`array(data)`	create numpy array
`base64_decode(s)`	decode base64 string
`base64_encode(s)`	encode string to base64
`bool(v)`	coerce to boolean
`ceil(n)`	round up to integer
`compress_zh(text)`	dictionary-code text using CJK symbols
`compress_zlib(text)`	compress text with zlib
`contains(s, sub)`	test substring presence
`cos(n)`	cosine (radians)
`cwd()`	current working directory
`date()`	current date (YYYY-MM-DD)
`datetime()`	current ISO timestamp
`decompress_zh(payload)`	inverse of compress_zh
`decompress_zlib(payload)`	inverse of compress_zlib
`dot(a, b)`	dot product of two vectors
`endswith(s, suffix)`	test string suffix
`env(name, default?)`	read environment variable
`exists(path)`	file or directory exists?
`float(v)`	coerce to float
`floor(n)`	round down to integer
`flow_add(f, id, type, cfg)`	add a block to flow
`flow_branch(f, cond, t, f)`	connect condition branches
`flow_connect(f, a, b)`	add default edge
`flow_new(name)`	create empty flow
`flow_register(f, name, fn)`	register custom action handler
`flow_run(f, initial?)`	execute flow
`flow_set_start(f, id)`	override start node
`flow_to_json(f)`	serialize flow structure
`flow_trace(f)`	get last execution trace
`gen_password(len?)`	random password
`gmatch(pattern, text)`	all regex matches
`gsub(text, pat, repl)`	regex substitution
`hex_decode(s)`	decode hex string
`hex_encode(s)`	encode string to hex
`int(v)`	coerce to integer

`join(list, delim)`	join list into string
`json_decode(text)`	parse JSON string
`json_encode(value)`	serialize value to JSON
`keys(dict)`	list of dictionary keys
`kmeans(data, k)`	k-means clustering
`len(collection)`	element count
`linear_regression(x, y)`	fit a linear model
`log(n, base?)`	natural or base-N log
`log10(n)`	base-10 log
`lower(s)`	lowercase copy
`ls(path?)`	list directory
`match(pattern, text)`	first regex match
`matmul(a, b)`	matrix multiplication
`max(...)`	maximum of args or list
`md5(text)`	MD5 hash
`mean(list)`	arithmetic mean
`memory_forget(mem, key)`	delete an entry
`memory_new(path?)`	create memory store
`memory_recall(mem, key)`	retrieve a value
`memory_save(mem, key, v)`	persist a value
`memory_stats(mem)`	count and size of entries
`min(...)`	minimum of args or list
`mkdir(path)`	create directory
`normalize(data)`	L2-normalize rows
`ones(n)`	array of ones
`pca(data, n)`	principal component analysis
`pi()`	π constant
`pop(list)`	remove and return last element
`pow(base, exp)`	exponentiation
`print(...)`	print args then newline
`println(...)`	alias of print
`push(list, item)`	append item to list
`quantum_decode(enc)`	inverse of quantum_encode
`quantum_encode(data)`	amplitude encoding
`quantum_measure(state)`	sample from $\lvert amp \rvert^2$ distribution
`random(a?, b?)`	random integer in [a, b]
`random_array(n, seed?)`	array of random values
`range(start, end, step?)`	integer range as list
`read(path)`	read entire file
`relu(x)`	ReLU activation
`replace(s, old, new)`	string replace
`reverse(list)`	reversed copy of list
`rm(path)`	delete file or directory
`round(n, decimals?)`	round to N decimal places

`sha1(text)`	SHA-1 hash
`sha256(text)`	SHA-256 hash
`sha512(text)`	SHA-512 hash
`sigmoid(x)`	sigmoid activation
`similarity(a, b)`	cosine similarity
`sin(n)`	sine (radians)
`sleep(seconds)`	pause execution
`softmax(x)`	softmax probability distribution
`sort(list)`	sorted copy of list
`split(s, delim)`	split string into list
`sqrt(n)`	square root
`startswith(s, prefix)`	test string prefix
`std(list)`	standard deviation
`str(v)`	coerce to string
`sub(s, start, end?)`	string slice
`sum(list)`	sum of numeric list
`svd_compress(matrix, rank)`	truncated SVD
`table(...)`	make dict from alternating k, v pairs
`tan(n)`	tangent (radians)
`time()`	unix timestamp
`token(length?)`	random URL-safe token
`trim(s)`	strip leading/trailing whitespace
`typeof(v)`	type name as string
`upper(s)`	uppercase copy
`url_decode(text)`	URL percent-decode
`url_encode(text)`	URL percent-encode
`uuid()`	random UUID as hex string
`values(dict)`	list of dictionary values
`vector_add(store, text)`	embed and index text
`vector_create(dim?)`	create empty vector store
`vector_search(store, q, k?)`	top-k nearest matches
`write(path, text)`	write text to file
`zeros(n)`	array of zeros

D Deployment & Troubleshooting

This appendix collects practical information about running BoRLang in production-like settings and resolving common problems. It is deliberately broad and opinionated rather than exhaustive; think of it as the notes a sysadmin might keep pinned to a monitor.

D.1 Running on a server

BoRLang itself is just a Python file, so any environment that can run Python 3.10+ can run BoRLang. For long-lived agents, the practical concerns are process supervision, logging, and secrets management. The following template covers the first two.

Create a systemd service file at **/etc/systemd/system/my-agent.service**:

```
[Unit]
Description=BoRLang agent
After=network.target

[Service]
Type=simple
User=agent
WorkingDirectory=/opt/borlang
ExecStart=/usr/bin/python3 /opt/borlang/borlang.py /opt/borlang/agent.bor
Restart=on-failure
RestartSec=10
StandardOutput=append:/var/log/my-agent.log
StandardError=append:/var/log/my-agent.err

[Install]
WantedBy=multi-user.target
```

Enable and start the service:

```
sudo systemctl daemon-reload
sudo systemctl enable my-agent
sudo systemctl start my-agent
sudo systemctl status my-agent
```

For agents that should run periodically rather than continuously, a cron entry is simpler:

```
# /etc/cron.d/my-agent
0 */6 * * * agent cd /opt/borlang && python3 borlang.py agent.bor
```

D.2 Secrets and credentials

Never hard-code API keys, passwords, or tokens in a BoRLang script. Use the **env** built-in to read them from environment variables, which can be set by your deployment system.

```
api_key = env("MY_API_KEY")
if api_key == nil
    print("Error: MY_API_KEY not set")
    return
end

flow_add(f, "call", "http", {
    url: "https://api.example.com/v1/data",
    headers: {Authorization: "Bearer " + api_key}
})
```

For systemd, set environment variables in the service file:

```
[Service]
Environment="MY_API_KEY=xxxx"
# Or, better, load from a protected file:
EnvironmentFile=/etc/my-agent/secrets.env
```

WARNING. The **secrets.env** file should have permissions 0600 and be owned by the service user, not checked into version control, and excluded from backups that leave the host. Consider using a secrets manager (HashiCorp Vault, AWS Secrets Manager) for anything production-grade.

D.3 Monitoring agent health

Every long-running agent should emit a heartbeat. The simplest scheme is to have the agent touch a file periodically; external monitoring checks the file's modification time and alerts if it gets stale.

```
def heartbeat()
    write("/var/run/my-agent.heartbeat", datetime())
end

# At the top of your main loop
while true
    heartbeat()
    # ... do work ...
    sleep(60)
end
```

A monitoring rule then checks that the file was touched within the last five minutes and raises an alert otherwise. This catches both crashes (no file update) and infinite loops (file updates but no other progress).

D.4 Common errors and fixes

ModuleNotFoundError: No module named numpy

The minimum dependencies were not installed. Run **pip install numpy scikit-learn** in the same environment where you run BoRLang. If you have multiple Python installations, verify you are using the right one with **which python3** and **python3 -m pip install**.

llm node 'x' failed: could not reach Ollama at ...

The **llm** block tried to connect to Ollama and got no response. Check that Ollama is running (**ollama serve**), that the **host** config matches the port it is listening on (default 11434), and that a model is pulled (**ollama list**). If Ollama is running on a different machine, set the **host** config to its address explicitly.

SyntaxError near 'end'

Usually means you have unbalanced block terminators. Each **if**, **while**, **for**, and **def** needs exactly one **end**. Count them. A common mistake is forgetting the **end** on a nested if, so the outer block consumes the wrong one and the parser reports the error at a confusing line.

RuntimeError: maximum recursion depth

A recursive function exceeded Python's stack limit (default about 1000 frames). Either rewrite iteratively, or — if you really need deep recursion — call **sys.setrecursionlimit** from a Python wrapper before invoking the interpreter. But iteration is almost always a better choice.

KeyError: 'X'

A dictionary lookup failed because the key does not exist. Either check with **keys(d)** first, or use the pattern **v = d[k]** followed by **if v == nil**. BoRLang lists raise on out-of-range index but dictionaries return nil from missing keys.

Flow seems to hang forever

Usually an HTTP or LLM block with no timeout set. BoRFlow defaults to reasonable timeouts (10 seconds for HTTP, 60 for LLM) but if you overrode them, a slow server can freeze the whole flow. Always set conservative timeouts in production and handle the timeout error with a condition block.

D.5 Frequently asked questions

Is BoRLang production-ready?

For prototypes, internal tools, and small agents — yes. For high-throughput, high-availability systems handling user traffic — not yet. The interpreter has no JIT, the standard library prioritizes correctness over throughput, and some edge cases in the parser are known and documented but not yet fixed. Version 3.2 plans a performance pass.

Can I embed BoRLang in another Python app?

Yes. Import the **Interpreter** class from the **borlang** module, create an instance, and call its **run** method with BoRLang source code. The interpreter has no state between runs unless you reuse the same instance, in which case global variables persist.

```
from borlang import Interpreter

i = Interpreter()
i.run('x = 42')
i.run('print(x + 8)')   # prints 50
# Access globals from Python
print(i.globals['x'])
```

Why not use Python directly?

You can and should, for most purposes. BoRLang is useful when you want a restricted sandbox language (for user-scriptable plugins, for example), when you want a single file that bundles language plus workflow engine (rather than Python plus LangChain plus one of several dependency graphs), or when the specific combination of Python-plus-Lua syntax is more teachable to your audience than plain Python.

Why not use LangChain?

Use LangChain if you need the ecosystem — hundreds of integrations, vector-store adapters, memory classes, and so on. Use BoRFlow if you want a small, understandable engine that you can read cover to cover, modify to suit your needs, and run without pinning a dozen package versions. The engines are not competitors so much as different points in the design space.

Does BoRLang support async?

Not directly in version 3.1. Flows run sequentially and HTTP/LLM blocks are blocking. If you need concurrency, use Python threading via an **action** handler, or run multiple BoRLang processes and have them communicate through the memory store or a shared file. Native concurrency primitives are planned for version 3.3.

How do I share a flow with a collaborator?

The flow definition and its calling code are just BoRLang source; share the **.bor** file. For purely-visual handoff to someone who does not program, export the flow as JSON with **flow_to_json** and open it in **flow_designer.html**, which renders the node graph interactively.

Where is the source code?

The project is bundled as the **borlang_v3_1.zip** archive. Unpacked, it is about 45 KB of Python. The core interpreter (**borlang.py**) is 700 lines and fits on a monitor. Read it if you want to understand how BoRLang actually works; the code is deliberately straightforward and heavily commented.

D.6 Where to go from here

If you have read this manual cover to cover and worked through the five tutorial projects, you know as much about BoRLang as anyone else. The best next step is to use it for something real: a small tool you actually need, not another tutorial. Nothing cements what you have learned like a weekend project that ships.

If you want to contribute back, the highest-impact areas for contributions are: new standard-library functions (especially domain-specific ones — time series, graph algorithms, image utilities), new BoRFlow block types that wrap common APIs, and improvements to error messages in the parser. Open a pull request on the official repository with your changes and a short description of why they are useful.

Bug reports are also valuable. If something fails where this manual says it should work, it is a bug in either the code or the documentation, and filing an issue helps everyone. Include the minimal input that triggers the bug, the expected output, and what you got instead.

The end. Thank you for reading.

E Language Design Notes

This appendix is optional reading. It explains why BoRLang makes the design decisions it does, comparing each with alternatives from other languages. It is intended for readers who want to understand the tradeoffs behind the syntax and semantics, or who are thinking of designing their own small language and want a worked example to argue with.

E.1 Why 'end' instead of indentation

Python's indentation-as-syntax rule is beloved by its users but hostile to naive text handling. Indented code breaks when pasted between editors with different tab settings, when passed through tools that normalize whitespace, and when embedded in other formats (JSON strings, shell scripts, email). Lua, Ruby, Pascal, and Algol all solve the same problem with explicit block terminators, and BoRLang joins them. The **end** keyword is slightly more verbose than Python's dedent, but it makes every program robust against whitespace corruption and easier to generate programmatically.

The word **end** was chosen over **done** (which shell uses), **fi/rof/enddo** (which Algol family languages use), and **}** (which C family languages use) because it is short, unambiguous in spoken English, and has a long track record in Lua and Ruby. It also avoids symbol pressure on the reader: a BoRLang program has no closing braces and minimal punctuation, which tends to feel calmer than a comparable Python or JavaScript program.

E.2 Why dynamic typing

BoRLang is dynamically typed. Every value carries its type at runtime, and type checks happen at operation time rather than at compile time. This is the same choice Python, Lua, Ruby, and JavaScript make. The advantage is simplicity: small programs need no type annotations, the interpreter can be smaller, and quick sketches stay quick to write. The disadvantage is that many classes of bugs only surface at runtime, sometimes in hard-to-reach code paths.

For BoRLang's intended use case — short scripts, prototypes, and agent definitions that change frequently — dynamic typing is the right choice. Programs are rarely long enough for type errors to hide, and the iteration speed gained from skipping annotations is valuable. A BoRLang 4.x with optional type annotations (like Python's **typing** module) is under consideration, but only as a late-stage hardening feature for code that has grown beyond its original scope.

E.3 Why no classes

BoRLang has no class or object-oriented-programming syntax. Many languages the size of BoRLang include them (Lua, JavaScript, Python), and at first glance this looks like an omission. The reasoning is that **closures** already provide everything classes give you at this scale — state encapsulation, polymorphism through first-class functions, and modular composition — with less syntactic machinery. Chapter 4 showed how a closure can carry state across calls; a dictionary of closures is, for all practical purposes, an object.

This decision was influenced by Scheme, whose designers made the same call: any time you are tempted to add classes, first see how far you can get with closures. For small programs the answer is always "further than you would expect". For very large programs the answer is "not quite far enough", but BoRLang is not aimed at very large programs.

E.4 Why flow-based agents

The current mainstream approach to LLM agents is the ReAct loop: a single prompt describes the available tools, the model decides which to call, the result goes back into the prompt, and the loop continues until the model decides it is done. This is elegant and minimal but has three weaknesses: the model is in control of orchestration (so you cannot easily enforce business logic), errors compound across steps (a wrong tool choice early on leads to a wrong answer later), and debugging requires inspecting token-level reasoning rather than step-level state.

BoRFlow takes the opposite approach: the developer explicitly describes the DAG of operations, and the LLM is just one operation among many. This gives up some of ReAct's flexibility — the agent cannot surprise you with a creative path through the tools — but gains predictability, testability, and inspectability. For production agents doing well-defined tasks, this is almost always the better tradeoff. For open-ended research agents, ReAct still wins.

Future versions of BoRFlow may add a block type that wraps a small ReAct loop, letting you mix both patterns in one flow. The explicit DAG handles the known structure; the embedded ReAct handles the open-ended parts. This combination is on the roadmap for version 3.4.

E.5 Why not compile to Python

BoRLang is interpreted by a tree-walking evaluator. Each expression is evaluated by recursing through its AST at runtime. This is the slowest possible way to implement a language; a compile step to Python bytecode, or to native code via LLVM, would speed things up by one or two orders of magnitude. Why not do that?

Three reasons. First, performance has not been a bottleneck in any reported use case — the interpreter is fast enough for anything interactive, and flow-level workloads are dominated by LLM and HTTP latency anyway. Second, tree-walking is the simplest possible implementation, which matters for a teaching language: the interpreter fits in 700 lines of Python, a readable afternoon for a curious student. Third, a compilation step would require a runtime library, complicating packaging and deployment. These tradeoffs may change in version 4.x, but for now the tree-walker stays.

E.6 Why Python as the host language

BoRLang's interpreter is written in Python. Alternatives considered were Rust (for performance and single-binary distribution), Go (for concurrency and simple tooling), and JavaScript (for browser deployment). Python won for a simple reason: the ML stack BoRLang exposes is already in Python. Writing the interpreter in any other language would require either a Python subprocess bridge (slow, fragile) or reimplementing NumPy and scikit-learn (infeasible).

The downside is that BoRLang inherits Python's startup time, global-interpreter-lock concurrency model, and dependency-management awkwardness. For the current use cases these costs are acceptable. If BoRLang ever grows into a domain where standalone binaries matter (embedded systems, CLI tools distributed to end users) a Rust port with FFI into Python for ML would be the natural next step.

E.7 What is deliberately missing

A short list of features BoRLang chooses not to have, each with a reason.

Exceptions. BoRLang uses return values for error signaling. A function returns **nil** or a dictionary with an **error** key; the caller checks explicitly. This is more verbose than try/except but keeps control flow visible at every level. Adding exceptions is easy to defer; removing them once people rely on them is very hard.

Generators and iterators. Python's **yield** keyword and the lazy-evaluation machinery behind it are powerful but add significant implementation complexity. BoRLang's collections are all eager — if you want laziness, write a state-carrying closure that returns the next value on each call. This is clunkier than **yield** but rarely needed in the kinds of programs BoRLang is designed for.

Macros. No compile-time metaprogramming. If you need to generate code, generate it as a string and **eval** it (well, there is no **eval** yet either, but you get the idea). Macros are a famously easy feature to over-use, and the absence forces clearer designs.

Interfaces and protocols. No way to declare "this function takes anything that responds to the **read** method". BoRLang uses duck typing: if the call works, it works. This matches the dynamic-typing decision and keeps the language small.

Pattern matching. No **match** or **case** statement. Nested **if/elif** handles every case that pattern matching would, with slightly more code. The omission keeps the parser simple and avoids the difficult design decisions around pattern syntax.

E.8 What the future looks like

Here is a rough sketch of what BoRLang might contain in five years. None of this is promised; it is speculative and depends on what users actually need.

Version 3.2 adds break/continue in loops, a match statement, fuller regex support, and a real network module. It also adds **flow_from_json** to round-trip flows through JSON. These are small, high-value additions that do not change the language character.

Version 3.3 addresses concurrency: native thread spawning, a message-passing primitive for agent coordination, and streaming support in the **llm** block. This is when BoRLang starts to feel like a proper production-agent platform rather than a teaching language.

Version 3.4 introduces optional static types, a bytecode compiler for hot paths, and a more sophisticated standard library around time-series analysis and document processing. This is hardening — the language feature set stabilizes and the implementation becomes robust.

Version 4.0 is a large enough release to reconsider decisions. Classes are on the table. A native package manager is on the table. A language server for IDE integration is definitely on the table. If BoRLang has reached version 4 it means people are using it for things that benefit from these features, and the language will have earned them.

The guiding principle through all these versions is the honesty principle that motivated the rewrite from version 2 to version 3: **ship what works, document what does not**. Every new feature must earn its place by solving a real problem; every promise in the documentation must be backed by a test; every limitation must be spelled out rather than hidden. If that principle holds, BoRLang will stay

useful regardless of what features it eventually accretes.

The End

— Milano, 2026 — L.L.B && D.B —

F Patterns Cheat-Sheet

This final appendix is a collection of short, self-contained code patterns. Each one solves a recurring problem that comes up when writing real BoRLang programs. Use the cheat-sheet as a reference: flip to the pattern you need, copy it into your code, adapt it to your specific case. The patterns are grouped by theme — data manipulation, flow construction, error handling, and I/O — but within each group they are presented in no particular order.

F.1 Data manipulation patterns

Safe dictionary default. Reading a key that might not exist without crashing.

```
v = d["maybe"]
if v == nil
    v = "default"
end
```

Increment counter. Bump a count for a key that may or may not already exist.

```
c = counts[key]
if c == nil
    counts[key] = 1
else
    counts[key] = c + 1
end
```

Group by. Partition a list of records into buckets by some field.

```
buckets = {}
for item in items
    k = item["category"]
    if buckets[k] == nil
        buckets[k] = []
    end
    push(buckets[k], item)
end
```

Deduplicate a list while preserving order of first occurrence.

```
seen = {}
unique = []
for x in items
    if seen[x] == nil
        seen[x] = true
        push(unique, x)
    end
```

```
end
```

Find max by key. Pick the element of a list with the largest value on some field.

```
best = items[0]
for x in items
    if x["score"] > best["score"]
        best = x
    end
end
```

Flatten list of lists. Concatenate nested lists into a single flat list.

```
flat = []
for sub in nested
    for x in sub
        push(flat, x)
    end
end
```

Zip two lists. Pair up elements from two parallel lists into a list of pairs.

```
pairs = []
i = 0
while i < len(a) and i < len(b)
    push(pairs, [a[i], b[i]])
    i = i + 1
end
```

Filter. Keep only elements matching a predicate.

```
kept = []
for x in items
    if x["active"] and x["score"] > 10
        push(kept, x)
    end
end
```

Map. Transform every element through a function.

```
result = []
for x in items
    push(result, transform(x))
end
```

Reduce. Fold a list to a single value.

```
total = 0
for x in numbers
    total = total + x
end
```

Invert a dictionary. Swap keys and values, assuming values are unique.

```
inverted = {}
for k in keys(d)
    inverted[d[k]] = k
end
```

F.2 Flow construction patterns

Linear pipeline. Three blocks in a row, each feeding into the next.

```
f = flow_new("pipeline")
flow_add(f, "a", "http", {url: "..."})
flow_add(f, "b", "transform", {field: "a.body"})
flow_add(f, "c", "print", {message: "got {{b}}"})
flow_connect(f, "a", "b")
flow_connect(f, "b", "c")
```

Two-way branch. Run one of two blocks depending on a condition.

```
flow_add(f, "check", "condition", {
    left: "{{score}}", op: ">", right: 0.5
})
flow_add(f, "high", "print", {message: "confident"})
flow_add(f, "low", "print", {message: "uncertain"})
flow_branch(f, "check", "high", "low")
```

Guard. Validate a precondition before running the main logic.

```
flow_add(f, "guard", "condition", {
    left: "{{status}}", op: "==", right: 200
})
flow_add(f, "abort", "print", {message: "bad status"})
flow_add(f, "proceed", "transform", {field: "body.data"})
flow_branch(f, "guard", "proceed", "abort")
```

Batch processing. Apply the same transformation to every element of a list.

```
flow_add(f, "process", "transform", {
    input: "item", operation: "upper"
})
flow_add(f, "loop", "loop", {
    over: "inputs", body: ["process"]
})
flow_set_start(f, "loop")
```

Fan-out then combine. Run several sub-flows and merge their results. Currently sequential; parallel execution lands in version 3.3.

```
results = []
for sub in [flow_a, flow_b, flow_c]
    r = flow_run(sub, shared_input)
    push(results, r)
end
# Now combine results as needed
```

Retry on failure. Call a flow up to N times until it succeeds.

```
attempt = 1
while attempt <= 3
    r = flow_run(f, ctx)
    if r["status"] == "ok"
        return r
    end
    sleep(attempt * 2)
    attempt = attempt + 1
end
```

Chained LLM calls. One prompt's output feeds into the next.

```
flow_add(f, "draft", "llm", {
```

```
    model: "qwen2.5:7b",
    prompt: "Write a rough outline for: {{topic}}"
})
flow_add(f, "polish", "llm", {
    model: "qwen2.5:14b",
    prompt: "Rewrite this outline more concisely: {{draft.response}}"
})
flow_connect(f, "draft", "polish")
```

F.3 Error handling patterns

Nil check after lookup. The universal "did this work" pattern.

```
result = read(path)
if result == nil
    print("Could not read " + path)
    return
end
# safe to use result
```

HTTP status gate. Only proceed if the call succeeded.

```
if result["status"] != 200
    print("HTTP failed: " + result["status"])
    return nil
end
body = result["body"]
```

Validate JSON from LLM. Models sometimes wrap JSON in markdown fences or add preamble. Strip both before parsing.

```
raw = trim(llm_output)
if startswith(raw, "```")
    raw = gsub(raw, "```json", "")
    raw = gsub(raw, "```", "")
    raw = trim(raw)
end
data = json_decode(raw)
if data == nil
    print("LLM returned invalid JSON")
    return
end
```

Timeout guard. Set conservative timeouts on every network call.

```
flow_add(f, "fetch", "http", {
    url: "...",
    timeout: 10        # seconds
})
flow_add(f, "ask", "llm", {
    model: "qwen2.5:14b",
    prompt: "...",
    timeout: 60
})
```

Circuit breaker. Stop hitting a service after repeated failures.

```
failures = 0
def call_service(input)
    if failures >= 5
        print("Circuit open, skipping")
        return nil
    end
    r = flow_run(service_flow, {input: input})
    if r["status"] != "ok"
        failures = failures + 1
        return nil
    end
    failures = 0
    return r
end
```

F.4 I/O patterns

Line-by-line read. Process a text file one line at a time.

```
content = read(path)
for line in split(content, "\n")
    cleaned = trim(line)
    if len(cleaned) > 0
        # process cleaned
    end
end
```

Append-only log. Record events to a growing file.

```
def log(event)
    line = datetime() + " | " + event + "\n"
    append("events.log", line)
end
```

Atomic write. Write to a temporary file first, then rename.

```
def atomic_write(path, content)
    tmp = path + ".tmp"
    write(tmp, content)
    # Rename is atomic on POSIX
    # (need a rename builtin in v3.2)
end
```

JSON file round-trip. Load a JSON file, modify it, save it back.

```
raw = read("config.json")
cfg = json_decode(raw)
cfg["last_run"] = datetime()
write("config.json", json_encode(cfg))
```

Directory walk. Process every file in a folder that matches an extension.

```
for f in ls("./docs")
    if endswith(f, ".txt")
        content = read("./docs/" + f)
        # process content
    end
end
```

Cached computation. Avoid recomputing expensive results between runs.

```
mem = memory_new("cache.db")

def expensive(input)
    key = sha256(input)
    hit = memory_recall(mem, key)
    if hit != nil
        return hit
    end
    result = actually_compute(input)
    memory_save(mem, key, result)
    return result
end
```

F.5 Agent patterns

RAG skeleton. Retrieve, augment, generate — the canonical pattern.

```
def rag(question)
    hits = vector_search(kb, question, 3)
    context = ""
    for h in hits
        context = context + h["text"] + "\n---\n"
    end
    f = flow_new("answer")
    flow_add(f, "reply", "llm", {
        model: "qwen2.5:14b",
        prompt: "Context:\n" + context +
                "\n\nQuestion: " + question
    })
    return flow_run(f)["reply"]["response"]
end
```

Critic-refiner loop. Have one LLM produce output, a second evaluate it, iterate.

```
def generate_with_critic(task)
    draft = generate(task)
    i = 0
    while i < 3
        review = critique(draft)
        if contains(lower(review), "good")
            return draft
        end
        draft = refine(draft, review)
        i = i + 1
    end
    return draft
end
```

Tool-use dispatcher. Let the LLM pick which tool to invoke by returning a keyword.

```
def dispatch(request)
    intent = classify_intent(request)
    if intent == "search"
        return search_tool(request)
    elif intent == "calculate"
        return calc_tool(request)
    elif intent == "remember"
        return memory_tool(request)
    else
        return default_response(request)
    end
end
```

Conversation summarization. Compress a long session into a paragraph to keep within context window.

```
def summarize_session(turns)
    transcript = ""
    for t in turns
        transcript = transcript + t["role"] + ": " +
                     t["text"] + "\n"
    end
    f = flow_new("summ")
    flow_add(f, "s", "llm", {
        model: "qwen2.5:7b",
```

```
        prompt: "Summarize this conversation in 3 sentences:\n" +
                transcript
    })
    return flow_run(f)["s"]["response"]
end
```

Self-consistency voting. Run the same prompt N times, pick the most common answer.

```
def vote(prompt, n)
    counts = {}
    i = 0
    while i < n
        ans = trim(lower(single_call(prompt)))
        c = counts[ans]
        if c == nil
            counts[ans] = 1
        else
            counts[ans] = c + 1
        end
        i = i + 1
    end
    # pick the key with highest count
    best = nil
    best_c = 0
    for k in keys(counts)
        if counts[k] > best_c
            best = k
            best_c = counts[k]
        end
    end
    return best
end
```

Chain with memory write-through. Every flow run persists its result for later retrieval.

```
mem = memory_new("history.db")

def run_and_remember(input)
    ctx = flow_run(agent, {input: input})
    key = "run_" + time()
    memory_save(mem, key, {
        input: input,
        output: ctx["final"],
        at: datetime()
    })
    return ctx["final"]
end
```

F.6 Testing patterns

Simple assertion. No native assert statement — roll your own.

```
def assert_eq(actual, expected, label)
    if actual == expected
        print("  OK   " + label)
    else
        print("  FAIL " + label)
        print("       expected: " + expected)
        print("       got:      " + actual)
    end
end
```

Test suite runner. Collect named test functions and run them all.

```
def test_addition()
    assert_eq(2 + 2, 4, "addition")
end

def test_concat()
    assert_eq("a" + "b", "ab", "concat")
end

tests = [test_addition, test_concat]
for t in tests
    t()
end
```

Stub LLM for tests. Replace real inference with deterministic responses during testing.

```
def make_stub_flow(response)
    f = flow_new("stubbed")
    def stub(ctx)
        return {response: response}
    end
    flow_register(f, "mock", stub)
    flow_add(f, "ask", "action", {name: "mock"})
    return f
end
```

— End of cheat-sheet —

That is every pattern I have found useful across the BoRLang projects I have built. Clip the ones you need, adapt them to your context, and build something interesting. Happy hacking.

◆ L.L.B && D.B · Milano · 2026